VFREFINE: DESIGN INTENT-DRIVEN VECTOR GLYPH OPTIMIZATION VIA SVG CODE REWRITING

Abstract

Large-scale Chinese font production is widely constrained by redundant vector glyph curves and irregular topology, which substantially increase storage and editing costs. To address this issue, this paper presents VFRefine, a vector glyph refinement framework based on a multimodal large language model. The framework supports both zero-shot refinement and few-shot reference-guided refinement. It aims to overcome the limits of geometric fitting by reformulating Chinese vector glyph refinement as a structured SVG code rewriting task that follows design intent. To model complex glyphs accurately, VFRefine integrates a dynamic-resolution visual encoder with a hybrid attention mechanism and introduces a hierarchical SVG decoding strategy. This strategy decouples vector generation into path command prediction and command-conditioned parameter prediction. Supported by the self-constructed VFRefine-1.2M dataset, which contains 1.2 million pairs, and a three-stage progressive training scheme, VFRefine achieves both strong zero-shot general refinement and effective few-shot reference-driven refinement. Experiments show that it significantly outperforms existing methods in visual consistency, geometric accuracy, and control point compression efficiency, which demonstrates strong potential for industrial-scale font production.

A. RELATED WORK

A.1. Vector Font Generation

Existing vector font reconstruction methods generally fall into two categories, namely optimization-based methods and learning-based methods. The former usually approximate the target glyph through explicit geometric operations in a progressive manner. For example, VecFontSDF [43] mainly fits glyph contours by composing and adjusting multiple closed Bézier curves. DualVector [14] further introduces Boolean operations to parse sets of closed paths and characterize the internal structure of glyphs. The main advantage of this line of work is that it relies less on large-scale vector annotation data, which makes it applicable to geometric characters with relatively simple structures. However, when glyph topology becomes more complex, these methods often produce redundant paths, insufficiently smooth boundaries, and even jagged artifacts.

In contrast, learning-based methods place greater emphasis on end-to-end modeling with high-quality paired vector data, and their central goal is to perform sequence-level prediction directly. SVG-VAE [18] is among the earlier frameworks for sequence generation in SVG fonts. VecFusion [30] combines a cascaded diffusion model with a Transformer and enhances the modeling of diverse topological structures through denoising generation over hybrid discrete and continuous representations. Meanwhile, the DeepVecFont series [37, 38] jointly encodes image information and sequence features, and exploits the strength of Transformers in long-sequence modeling to achieve an effective mapping from pixel representations to drawing commands. Different from the above research line that centers on sequence generation, Song et al. [29] propose a component-assembly generation scheme, which predicts the affine transformation parameters of predefined vector radicals through a regression network to efficiently construct complex Chinese character structures.

Although the above studies have achieved clear progress in visual reconstruction quality, a substantial gap still remains between their outputs and the vector quality required in professional font design. The main reason is that most existing methods focus on visual consistency in pixel space, while paying insufficient attention to the geometric regularity, structural validity, and downstream editability of the vector paths themselves. As a result, generated outputs still commonly exhibit unordered anchor point distributions, broken paths, and local overlaps, which makes them difficult to use in high-quality font production. Motivated by this limitation, this paper no longer restricts the goal to vector glyph generation alone, but instead focuses on vector font optimization, that is, the intelligent reconstruction of non-canonical or noisy vector paths to obtain high-quality vector code with more compact geometric representations, clearer topological organization, and better editability for downstream design.

A.2. Vector Graphics Generation

In the early stage of vector graphic generation research, related work mainly focuses on representation modeling and learnable generation mechanisms. DeepSVG [5] exploits the hierarchical modeling capacity of Transformers to improve the representation of complex icon generation and interpolation. Im2Vec [24] attempts to use control points on a deformable unit circle as an intermediate representation to map raster images to vector drawing commands. Meanwhile, DiffVG [13] introduces differentiable rasterization, which establishes a gradient propagation path between vector graphics and bitmap space. This advance lays the foundation for subsequent methods that combine optimization and learning.

Driven by the above studies, an optimization-based generation paradigm built on differentiable rendering gradually emerges. CLIPDraw [7] is the first to leverage the semantic supervision provided by CLIP and directly optimize Bézier curve parameters in an iterative manner, thereby enabling text-to-vector graphic generation without explicit training data. Subsequently, to address the loose geometric structures that often arise when relying only on CLIP supervision, VectorFusion [10] introduces a score distillation sampling mechanism, which transfers the priors of pretrained pixel diffusion

models into the vector generation process and thus improves texture details and stylistic quality. However, optimization-centered methods of this kind still typically suffer from high inference cost, vulnerability to local optima, and limited control over topological structures.

To overcome the limitations of the optimization paradigm, subsequent studies turn to native generation methods centered on diffusion probabilistic models. Considering the complex nonlinear relationship between vector parameters and final visual appearance, Dual-domain Diffusion [48] builds a collaborative interaction mechanism between the pixel domain and the vector domain. Through cross-domain feature transfer, it alleviates the gradient disconnection issue and improves visual semantic expressiveness while preserving relatively regular geometric topology. In addition, given the discrete structure of SVG commands, Discrete Flow Matching [8] begins to jointly model topological structures and geometric parameters in discrete space, thereby broadening the modeling scope of vector generation.

In recent years, with the rapid development of multimodal large models, treating SVG as executable structured code gradually becomes a new research focus, namely the "SVG as Code" paradigm. In this direction, IconShop [42] employs an autoregressive Transformer to model serialized SVG tokens and demonstrates strong compositional generation ability. Going further, StarVector [25] reformulates vector graphic generation as a multimodal code generation task. By leveraging the advantage of large language models in syntactic reasoning, it produces SVG code with more complete structure and better editability. Chat2SVG [41], in contrast, proposes a hybrid framework in which an LLM handles high-level layout planning and a diffusion model refines geometric details, achieving a favorable balance between semantic organization and low-level geometric precision.

A.3. Multimodal Large Language Models

In recent years, large language models (LLMs) demonstrate strong capability in natural language processing and drive rapid progress across a range of fundamental tasks [12, 22, 31, 32, 35]. Their advantage is particularly evident in code generation, where they capture the syntactic constraints and structural patterns of formal languages effectively [3, 9, 20]. Building on this foundation, multimodal large language models (MLLMs) incorporate visual encoding modules to combine image perception with language modeling frameworks. As a result, models that originally focus on text processing extend to a variety of cross-modal tasks, including visual question answering, image captioning, text-to-image generation, and document understanding, and achieve substantial progress [12, 21, 23, 45, 11].

Nevertheless, early MLLM research remains largely focused on image understanding and content generation based on pixel representations. As a result, visual encoders are typically better at extracting global semantic information, yet relatively limited in capturing local geometric details and precise structural relations. This limitation makes them difficult to apply directly to engineering design tasks that require high geometric accuracy and structural consistency. To address this issue, recent studies begin to explore a new modeling paradigm that unifies visual perception and structured code generation within a single framework. Since SVG graphics are inherently organized in XML form and possess both visual expressiveness and textual descriptiveness, they provide a natural interface for connecting image content with symbolic structure. For this reason, compared with traditional vector generation methods that rely on explicit optimization, this line of work is more naturally formulated as a program synthesis problem conditioned on visual input.

As the ability to jointly model visual information and language output continues to improve, the scalability of MLLMs in complex multimodal applications becomes increasingly evident [46]. Recent studies, such as StarVector [25] and LLM4SVG [44], show that this long-standing modality conversion challenge between raster images and vector code can be effectively alleviated by adopting a visual encoder that is more sensitive to geometric patterns and combining it with a pretrained large code model as the generation backbone. With this design, the model not only understands the semantic content of an image but also directly outputs XML parameter representations that satisfy syntactic constraints, thereby enabling relatively precise reconstruction and generation of vector structures in the image.

A.4. SVG Datasets and Benchmarks

At present, the shortage of SVG data for non-canonical fonts becomes one of the major factors that constrain continued progress in this area. Existing Image-to-SVG datasets show a clear divide in their composition. One line of datasets focuses on general visual graphics and mainly covers objects such as illustrations, icons, and emojis [5, 42, 4]. For example, although SVG-Stack [25] contains about 2.1 million samples collected from real code repositories and preserves structural information at the raw syntax level well, its font-related content mostly consists of geometric glyphs built from standard primitives, with limited topological noise and path irregularities that are common in non-canonical fonts. MMSVG [47] is representative in terms of multimodal semantic alignment, yet the character subset it includes is mainly centered on anime figures. The construction logic of such graphics differs substantially from that of textual glyphs, which place much stronger emphasis on stroke continuity and structural constraints.

Another line of datasets targets standard printed fonts with strict structure and regular forms. SVG-Fonts [18], for instance, provides high-quality font samples with well-closed paths. However, because the data are overly idealized, they fail to reflect the hand-drawn deformation, local noise, and structural distortion that commonly arise in real design workflows. As a result, they are also less effective for supporting the learning of mappings from non-canonical inputs to canonical outputs. Meanwhile, for text-driven SVG generation, LLM4SVG [44] collects and normalizes about 250,000 vector graphics with rich colors and complex structures. This dataset substantially expands the available data foundation for related research and provides strong support for the Text-to-SVG setting. Overall, however, these data resources either serve general graphic generation or are constructed for specific tasks, and some of them are not fully public. They are therefore difficult to apply directly to the more targeted problem of high-precision vector path refinement for fonts. To fill this gap, we construct the VFRefine-1.2M dataset. It covers 200 fonts and contains 1.2 million non-canonical to canonical sample pairs, with the goal of providing more systematic

and task-specific data support for research on vector path refinement for non-canonical fonts.

B. DATASETS

Improvements in model performance depend to a large extent on the quality of the training data, yet most existing open-source data resources place greater emphasis on superficial visual appearance during construction and pay relatively limited attention to the topological regularity of vector graphics. To address this issue, we construct the large-scale VFRefine-1.2M dataset, which mainly targets typical defects such as redundant vector primitives and disordered structures. The dataset contains 1.2 million samples and characterizes the unoptimized vector states commonly observed in real design workflows, thereby providing sufficiently rich supervision for learning how to reconstruct low-quality vector representations into more compact and better-structured SVG code.

B.1. Data Construction Pipeline

The construction of VFRefine-1.2M combines authentic editing traces from manual design with sample characteristics generated under algorithmic control, allowing the overall data distribution to cover, to a sufficient extent, the diverse vector refinement issues commonly encountered in practical font engineering:

Real-world Defects from Design Archives.

To make the data better aligned with real industrial scenarios, we collaborate with Hanyi Font and systematically organize and mine its core design archives accumulated from 2017 to 2025. In the construction process, we treat the original vector drafts produced in the early creative stage by designers as the input sequences to be refined, and use the commercial final versions, after topological reorganization and geometric correction by the professional Hanyi team, as the corresponding standard target outputs. Based on this retrospective historical version pairing strategy, we establish a real-mapping subset containing 380,000 samples. Compared with data generated solely by procedural synthesis, this subset not only preserves the operation patterns and editing biases that are difficult to model explicitly in manual design, but also reflects the refinement experience embodied in the evolution of a font from an initial draft to an industrial-grade final product. Therefore, this portion of the data provides supervision signals with greater practical value and improves the model's ability to adapt to real font refinement tasks.

Algorithmic Synthetic Subset.

In addition to the data derived from real font design, we also construct a synthetic subset with approximately 820,000 samples to simulate the machine-generated defects commonly produced by existing automatic vectorization tools during glyph contour reconstruction. We introduce this portion of the data because many vectorization methods can recover the basic glyph shape from raster images, yet their outputs often suffer from irregular topology, redundant control points, and local structural distortions. To enable the model to learn and correct such typical defects, we take 200 commercially released fonts from the Hanyi font library, each having passed professional quality inspection, as standard references and design the following synthetic data generation pipeline:

1. **Rasterization:**First, we use FontTools [34] to extract high-quality standard vector glyphs from font files and render them into raster images with a resolution of 512×512, which serve as unified inputs for the subsequent automatic reconstruction stage.
2. **Vector Reconstruction:** To cover topological defects and geometric noise from different sources, we apply two representative vectorization methods to convert the raster images back into vector code and thereby generate sequences for optimization:
   - *Graphics-based Methods:*We use Potrace [28], VTracer [33], and AutoTrace [40], which are widely used graphics-based methods in industry. These methods usually approximate the original contours with a large number of short curve segments, and therefore tend to introduce overly dense control points, fragmented noise, and redundant path representations.
   - *Deep Learning-based Methods:* We also include deep learning models based on sequence prediction, including DeepVecFont [37], DeepVecFont-V2 [38], Dual-Vector [17], DiffVecFont [16], and VecFontSDF [43]. Compared with traditional graphics-based methods, these models show certain advantages in fitting the overall shape, but their outputs can still contain topological errors and unnecessary data redundancy.
3. **Data Filtering:** Since automatic reconstruction inevitably produces some invalid samples or severely distorted results, we further introduce a strict data filtering stage. Specifically, we measure the geometric alignment between each reconstructed result and the corresponding ground truth, and remove abnormal samples that exhibit significant structural differences or cannot be correctly parsed by a standard rendering engine, thereby ensuring the reliability and usability of the training data.
4. **Pair Construction:** Finally, we pair each low-quality vector result produced by the two types of methods above with its corresponding original high-quality vector glyph from the Hanyi font library. This construction yields a dataset with substantial scale and also enables the model to learn two key abilities under supervised learning, namely geometric redundancy removal and topological structure correction.

B.2. Dataset Organization and Splits

To maintain dataset scale while ensuring fair evaluation and sample diversity, VFRefine-1.2M adopts a strict font-level split strategy. The dataset covers 200 fonts in total, among which 160 are used for training, 20 for validation, and the remaining 20 for testing. Since the fonts in the validation and test sets never appear during training, this split more effectively evaluates the model's generalization ability to unseen font styles. The detailed composition of each subset is shown in Tab. 4:

- **Training Set:** The training set contains 800,000 samples and covers 160 font styles. Among them, about 580,000 samples come from algorithmic reconstruction results, which help the model learn broader geometric priors. At the same time, the set includes 220,000 real

designer drafts, accounting for about 27.50%, in order to preserve the artistic characteristics and editing habits that arise in real creative workflows.

- **Validation Set:** The validation set consists of 20 independent fonts and contains 200,000 samples in total, of which 30% are real drafts, that is, 60,000 samples. This proportion helps reflect more realistically how the model performs on industrial-grade real data during model selection and hyperparameter tuning.
- **Test Set:** The test set also contains 200,000 samples, all drawn from another 20 fonts that are unseen during training. To minimize the impact of distribution shift on evaluation results and ensure fair comparison across different methods, this paper strictly controls the ratio between real draft samples and algorithmic reconstruction samples to 1:1. Furthermore, the test set is divided into two parts, Seen Characters (100,000) and Unseen Characters (100,000). The former examines the model's ability to adapt to new font styles under known structural conditions, while the latter evaluates its reasoning and reconstruction ability when facing completely unfamiliar Chinese character topologies.

C. DATA PROCESSING AND CURATION

C.1. Scalable Vector Graphics

Scalable Vector Graphics (SVG) is an XML-based format for two-dimensional vector graphics. Structurally, an SVG document consists of the following primary components, which are here exemplified by data corresponding to the character "永" from the Heiti typeface:

1. XML Declaration. Typically situated at the beginning of the file, it specifies the XML version and encoding.

```
<?xml version="1.0" encoding="UTF-8" standalone="no"?>
```

2. Root Element `<svg>`. All graphical content is encapsulated within the `<svg>` tag. Common attributes include the xmlns namespace (typically set to http://www.w3.org/2000/svg), width and height, which specify the dimensions of the viewport, and viewBox, which defines the visible region within the user coordinate system using the format "minX minY width height".

```
<svg xmlns="http://www.w3.org/2000/svg"
width="512" height="512"
viewBox="0 0 512 512">
<path ></path>
</svg>
```

3. Path. In SVG, the `<path>` element is utilized to render complex shapes. The specific geometry is defined via the d attribute, which consists of a string composed of a sequence of commands and parameters that delineate the rendering trajectory. A summary of common commands and parameters is presented in Tab. 5.

```
<path d="M 204 230 Q 198 284 174 338 Q 151 392 114 428 Q 77
    465 41 488 Q 28 467 0 450 Q 56 424 95 380 Q 133 336 153
    262 L 75 262 Q 49 262 13 265 L 13 228 Q 49 230 75 230 Z
  "></path>
```

C.2. Vector Glyph Serialization

In the data representation stage, we first apply structured processing to Chinese glyphs composed of Bézier curves and convert them into SVG XML encoding in a unified manner. To standardize the description of the vector trajectories of individual strokes in each glyph, this paper uses the seven basic drawing commands defined by the SVG standard as the unified representation units, as shown in Tab. 5. Fig. 8(a) uses the Chinese character "永" as an example to illustrate how the vector geometric structure of a character is represented.

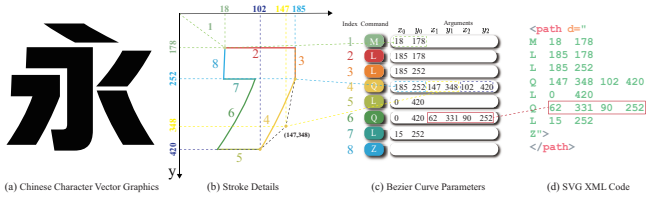


Fig. 8. (a) Vector image representation of a Chinese font character. (b) Illustration of how a single stroke is drawn in a two-dimensional coordinate system. (c) Correspondence between Bézier curve parameters and curve shapes. (d) XML representation based on absolute coordinates, which contains drawing commands and their corresponding coordinate information.

Specifically, Fig. 8(b) selects a representative stroke segment from the character "永" to illustrate the fine-grained control provided by Bézier curves in glyph construction. Its drawing process is analogous to the continuous movement of a pen tip during writing, where a sequence of commands is executed in order to control the state of the pen, thereby gradually generating the complete character outline.

As shown in the command table in Fig. 8(c), each row represents a discrete drawing step, where Index corresponds to the different colored path segments in Fig. 8(b), and these segments jointly form the complete stroke outline in sequential order.

Starting point positioning: The path first uses the move command `M` to lift the pen and move it to the absolute coordinate (18, 178), which establishes the starting position of the stroke (Index 1, corresponding to the starting point marked by the green dashed box).

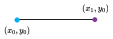
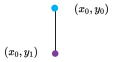
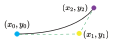
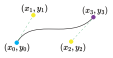
Straight-line connection: The path then uses the `L` command to connect the starting point to (185, 178) (Index 2), forming the first straight boundary segment. In the subsequent structure, Index 6, 5, and 7 are also defined by straight-line segments, which connect in sequence to (185, 252), (0, 420), and (15, 252), respectively, to represent the relatively flat portions of the stroke outline.

Quadratic Bézier curve construction: The main outer contour of this stroke is formed by a sequence of connected quadratic Bézier curves. Taking Index 4 as an example, the curve starts from the current point (185, 252), uses the control point (147, 348) to regulate the bending trend, and finally reaches the endpoint (102, 420). The subsequent Index 6 further refines the upper turning region with another quadratic Bézier segment: it starts from (0, 420), passes through the control point (62, 331), and ends at (90, 252),

Table 4. Statistical distribution of the VFRefine-1.2M dataset across different splits and data sources.

Split	Total Samples	Human Design Drafts	Algorithmic Reconstructions	Real Ratio
Training	800,000	220,000	580,000	27.50%
Validation	200,000	60,000	140,000	30.00%
Test	200,000	100,000	100,000	50.00%
Total	**1,200,000**	**380,000**	**820,000**	**31.67%**

Table 5. Vector glyphs consist of a different number of SVG drawing commands, with each drawing command representing a parameterized shape contour.

Commands	Arguments	Example	Description
M	x_1, y_1		Move from (x_0, y_0) to (x_1, y_1)
L	x, y		Draw a line from (x_0, y_0) to (x_1, y_1)
H	x_1, y_1		Draw horizontal line from (x_0, y_0) to (x_1, y_0)
V	x_1, y_1		Draw vertical line from (x_0, y_0) to (x_0, y_1)
Q	x_1, y_1 x_2, y_2		Draw quadratic Bézier curve from (x_0, y_0) to (x_2, y_2), control point (x_1, y_1)
C	x_1, y_1 x_2, y_2 x_3, y_3 x_4, y_4		Draw cubic Bézier curve from (x_0, y_0) to (x_3, y_3), control points (x_1, y_1) and (x_2, y_2)
Z	$\varnothing$		Close path from (x_0, y_0) to (x_0, y_1)

which keeps the contour smooth at the corner.

Path closure: Finally, the path uses the `Z` command (Index 8) to automatically connect the current point to the starting point, thereby forming a closed region and completing the contour definition of this stroke.

C.3. SVG-based Glyph Discretization

After completing the drawing commands above, the glyph information is further serialized into a standard XML text representation, as shown in Fig. 9. During this conversion, we apply coordinate discretization by uniformly rounding all floating-point drawing parameters to integers. This strategy is mainly motivated by two considerations, namely computational efficiency and glyph fidelity.

Reducing computational cost: In the tokenizer of an MLLM, floating-point numbers are usually split into multiple tokens, whereas integers with no more than three digits can usually be processed as a single token. Therefore, removing the fractional part significantly shortens the sequence length, which improves inference efficiency and increases the utilization of the context window. This compression is particularly important for glyph tasks with complex strokes and long text sequences.

Glyph consistency: SVG images are usually rendered with screen pixels as the basic unit. In glyph optimization at a resolution of 512×512, subpixel coordinate precision is unnecessary. After converting coordinates to integers, we remove small numerical perturbations that have limited effect on the final visualization, without compromising the integrity of the glyph contour, thereby preserving the overall consistency of the glyph.

D. DETAILED PAIRED DATA FORMULATION

Representative non-canonical and expert-refined glyph pairs are shown in Fig. 10; they preserve raster appearance while exhibiting substantially different SVG topology.

Unlike common computer vision tasks, where explicit annotations can indicate target regions directly, redundant control point identification in vector font optimization does not admit a straightforward annotation scheme. The control points of a Bézier curve are not independent of one another but are tied by strong structural relations. Once a control point is removed, the parameter configuration within its neighborhood also needs to be adjusted accordingly to prevent noticeable shifts in the glyph contour. Meanwhile, the choice of a more desirable topology usually lacks a unified and easily externalized criterion. Such judgments rely more on experience accumulated in font design, which makes them

Fig. 9. SVG-based discrete visualization of glyphs. Panels (a) and (b) compare SVG XML code with high-precision decimal coordinates and discretized integer coordinates, highlighting the substantial reduction in sequence length. Panels (c) and (d) show the corresponding rendered glyphs, while the overlay comparison in (e) shows that integer discretization strictly preserves the structural integrity and visual fidelity of the glyph while effectively filtering numerical noise without sacrificing quality.

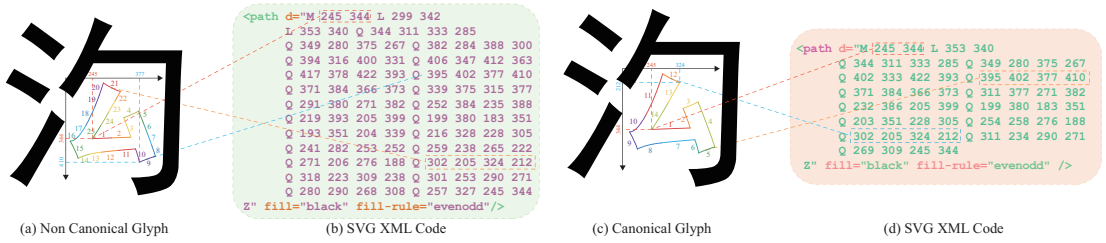


Fig. 10. Representative paired samples of non-canonical and canonical glyphs.

difficult to capture through simple and explicit annotations.

To address this issue, we do not follow the conventional discriminative annotation paradigm. Instead, we formulate the task as a generative transformation from a non-canonical representation to a canonical representation and construct parallel corpora on this basis. In other words, the annotation that is previously difficult to define explicitly is converted into a mapping between paired sequences. The model does not need to decide directly whether a local control point should be kept or removed. Rather, it learns the overall transformation process and thereby acquires the principles of normalization implicitly. This paired-sequence formulation therefore offers several advantages:

- **Overall topological supervision:** This strategy provides supervision over the overall topological structure rather than point-level annotations at the local level. Conventional local annotations cannot adequately reflect the global geometric constraints of a glyph, nor can they characterize the coupling between control point removal and curve shape preservation. By contrast, when the complete expert-optimized SVG code is used as the ground-truth target, the model learns not only how to eliminate redundant control points, but also how to reconfigure the remaining control points after topological adjustment so as to maintain curve continuity and smoothness. Therefore, this form of supervision is global and structural.

- **End-to-end semantic alignment:** This construction is also well aligned with an end-to-end sequence modeling framework. We transform what is originally a visually defined optimization problem into a conversion task that is more suitable for sequence modeling, where the input is an over-parameterized representation and the output is a compact and normalized representation. Through this paired sequence mapping, glyph optimization can be unified under a translation-like modeling framework, which allows the use of the autoregressive generation mechanism of MLLMs and their context modeling capacity to handle

complex geometric dependencies and structural relations that extend beyond local regions.

- **Avoiding subjective ambiguity :** This method also helps reduce annotation ambiguity caused by differences in subjective judgment. In font design practice, there is no fully consistent manual criterion for whether a given node should be removed. To address this issue, we use the final commercial release from Hanyi Fonts as the sole ground truth, so that the model objective is aligned directly with the design standard reflected in industrial-grade finished glyphs. This choice avoids label noise introduced by individual aesthetic differences during manual annotation and provides a more stable and consistent reference for model optimization.

As shown in Fig. 11, we present a pair of non-canonical and canonical samples from VFRefine-1.2M. Although the two are almost visually indistinguishable after rendering, as shown in Fig. 11(a), their underlying SVG sequences differ substantially. In this example, the non-canonical SVG contains 106 drawing commands across two subpaths, whereas the expert-level canonical SVG reduces this number to 52, which removes 54 commands in total and corresponds to a reduction of 50.94%. At the subpath level, the number of commands in the first subpath decreases from 50 to 28, which is a reduction of 44.00%, while the second subpath decreases from 56 to 24, which is a reduction of 57.14%. This result shows that, while preserving the same visual appearance, expert-level optimization effectively achieves control point sparsification and path structure reorganization, thereby producing a more compact vector representation, improving storage and rendering efficiency, and better satisfying the needs of industrial editing. Such paired data makes it possible to train VFRefine without relying on explicit geometric rules, allowing it to learn the transformation from draft to finished glyph in a data-driven manner.

E. DATA AUGMENTATION

To prevent the MLLM from merely memorizing superficial patterns in specific coordinate sequences and to improve its ability to adapt to different forms of topological organization, we design a data augmentation method that preserves glyph shape. This method exploits both the algebraic decomposability of Bézier curves and the representational equivalence of vector graphics geometry to generate equivalent sequences with substantially different code representations, while keeping the visual appearance of the glyph unchanged. As a result, it improves model robustness to diverse representation forms. This augmentation strategy proceeds from three main aspects:

E.1. Controlled Fragmentation of Bézier Curves.

To simulate the over-parameterization commonly observed in non-canonical glyphs, we construct negative samples that contain geometric redundancy. Specifically, we perform lossless subdivision on a standard Bézier curve. Given an nth-order Bézier curve $B(t)$ defined by the control point sequence $\mathbf{P} = \{P_0, P_1, \ldots, P_n\}$:

$$B(t) = \sum_{i=0}^{n} \binom{n}{i} (1-t)^{n-i} t^i P_i, \quad t \in [0,1]. \tag{15}$$

We randomly select a splitting point $u \in (0,1)$ in the parameter domain t and divide the curve into two consecutive sub-curves $B_1(t')$ and $B_2(t')$, where $t' \in [0,1]$. This operation is strictly equivalent in geometry, but at the sequence level, it replaces the original command with two new sets of control point parameters, thereby introducing controlled geometric redundancy. This process forces the model to learn the merging logic of Bézier curves, namely, to recover a compact parametric description from fragmented segments.

E.2. Cyclic Shifting of Closed Contours

Closed paths in vector glyphs exhibit cyclic invariance. For a closed sequence $\mathcal{S} = \{c_1, c_2, \ldots, c_K\}$ composed of K commands, changing the position of its starting command does not alter the rendering result. To remove the model's dependence on a specific starting point, we apply a random cyclic shift to closed paths during training:

$$\mathcal{S}' = \{c_j, c_{j+1}, \ldots, c_K, c_1, \ldots, c_{j-1}\}, \tag{16}$$

where j is a randomly selected starting index. This strategy ensures that the model focuses on the global topological structure of the closed contour rather than developing a memorization bias toward the absolute position of the first token in the sequence.

E.3. Order Shuffling of Independent Sub-paths

Chinese character glyphs are typically composed of multiple topologically independent sub-paths. We further exploit the commutativity of the drawing order of non-intersecting sub-paths in SVG rendering and introduce a path order augmentation method. Let the set of sub-paths corresponding to a glyph be $\mathcal{G} = \{S_1, S_2, \ldots, S_N\}$. Since the visual composition of these independent strokes is unaffected by their ordering, we sample from their permutations and randomly reorder the occurrence of sub-paths at the XML sequence level. This operation does not change the appearance of the final rendered glyph, but it reduces the model's reliance on specific stroke ordering patterns. In this way, the model places greater emphasis on learning stable spatial geometric relationships rather than memorizing a fixed order in the input sequence, which improves its ability to adapt to vector inputs produced under different design conventions and encoding organizations.

F. REFINEMENT PARADIGMS

The VFRefine framework includes two complementary paradigms for vector glyph optimization, **Few-Shot Reference-Guided** and **Zero-Shot Universal**. The former performs style-adaptive inference from standard reference samples, whereas the latter relies on the model's internalized structural priors to achieve general reconstruction of non-standard glyphs without explicit external references.

F.1. Few-Shot Reference-Guided

In the few-shot reference-driven setting, the key objective is to use a small number of expertly optimized standard samples to enable rapid adaptation to a specific font style and its corresponding complex topological structure. Specifically, given a target font style s, we select a small set of expertly

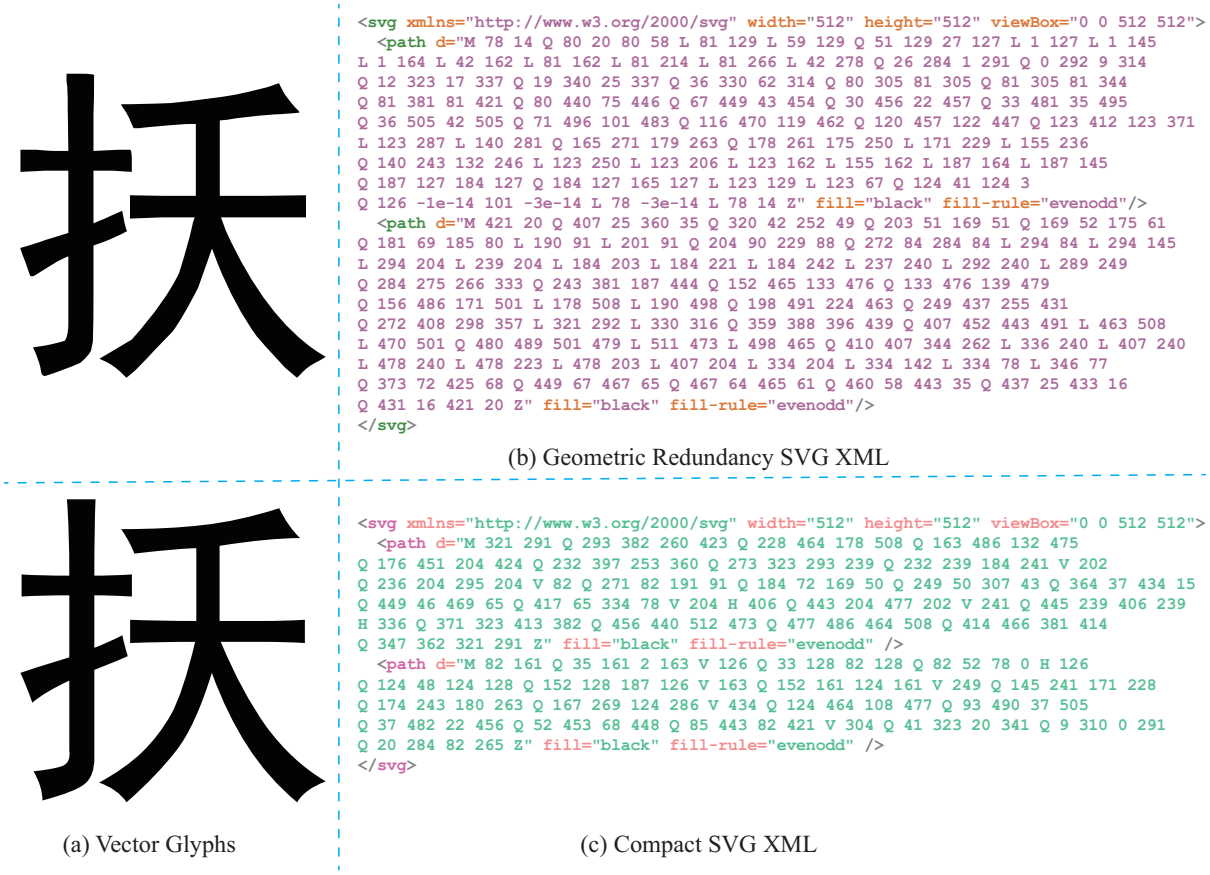


Fig. 11. Visualization of a paired data construction example. The figure shows a sample pair consisting of a non-canonical input and its corresponding expert-level optimized ground truth. (a) The rendered vector glyphs remain highly consistent in visual appearance. (b) The non-canonical sequence contains substantial geometric redundancy and densely distributed control points. (c) The canonical glyph is concise and compact.

Table 6. Instruction templates used for few-shot reference-guided and zero-shot universal vector refinement.

Type	Role	Content Template
# 1	SYSTEM	You are a helpful assistant. Your task is to refine non-canonical initial SVG code into industrial-grade standard code with compact topology. Please refer to the target style exemplar and adopt an optimal control point distribution strategy to remove geometric redundancy while strictly preserving visual fidelity.< /s >
	USER	Reference images: {IMG}, ..., {IMG}; reference canonical SVG:{SVG}, ..., {SVG}; source image: {IMG}; source non-canonical SVG:{SVG} < /s >
	ASSISTANT	SOV Path MoveTo Coord Bézier curve Coord LineTo Coord ... FILL ... EOV < /s >
# 2	SYSTEM	You are a helpful assistant. Your task is to refine non-canonical SVG code into compact code that meets industrial standards. Without external references, you independently infer the optimal topological structure of the input character. By accurately analyzing the visual features of the source image and removing the geometric redundancy of the original paths, you reconstruct an SVG sequence that is minimal, precise, and consistent with professional design standards.< /s>
	USER	Source image: {IMG}; source non-canonical SVG:{SVG} < /s >
	ASSISTANT	SOV Path MoveTo Coord Bézier curve Coord LineTo Coord ... FILL ... EOV < /s >

optimized character samples with complete and standardized topological structures from the glyph set of that style, and construct the reference set accordingly:

$$\mathcal{R}_s = \{(I_k^{\text{ref}}, S_k^{\text{ref}})\}_{k=1}^{K}, \tag{17}$$

where, I_k^{ref} denotes the raster image of the k-th reference character, and S_k^{ref} denotes its corresponding expert-standardized SVG description. For any source character $(I_{\text{src}}, S_{\text{src}})$ to be processed, the model jointly encodes $\mathcal{R}_s$ and $(I_{\text{src}}, S_{\text{src}})$ within a multimodal large language model (MLLM) framework. Through the attention mechanism, the model captures regularities in node layout and curve parameterization from the reference samples and transfers them to the optimization of the source character, ultimately generating the geometrically optimized target sequence S_{tgt}.

Under this task setting, the model aims to learn the following mapping:

$$\big(\mathcal{R}_s, I_{\text{src}}, S_{\text{src}}\big) \mapsto S_{\text{tgt}}. \tag{18}$$

Unlike conventional methods that rely on manual design or fit using only a single modality, our method centers on the in-context learning capability of the MLLM. Even with only a few reference samples, the model effectively infers the geometric organization patterns specific to the target style and accordingly performs vector reconstruction with consistent style and high precision. This process enables the model to rapidly adapt to the structural characteristics of a new style through few-shot in-context examples. The instruction template is shown in Tab. 6 #1.

F.2. Zero-Shot Universal

Unlike the few-shot task that relies on reference samples, the zero-shot general optimization task aims to directly infer an industrial-standard compact vector representation without any additional style references, using only the visual and geometric information of the input character itself. Under this setting, we use the VFRefine-1.2M dataset to construct a large-scale mapping:

$$\mathcal{D} = \{(I_i, S_i^{\text{src}}, S_i^{\text{tgt}})\}_{i=1}^{N}, \tag{19}$$

where, I_i and S_i^{src} denote the raster image and the non-canonical SVG representation of the i-th character, respectively, and S_i^{tgt} denotes the corresponding canonical target SVG. Based on the dataset $\mathcal{D}$, we train the optimization model to maximize the conditional generation probability:

$$P_\theta(S_{\text{tgt}} | I_{\text{src}}, S_{\text{src}}). \tag{20}$$

This process enables the model to internalize general design principles and topology optimization strategies from the global distribution over diverse fonts and styles.

At inference time, for any input source character $(I_{\text{src}}, S_{\text{src}})$, the model requires no external reference and directly uses the implicit structural priors learned during pretraining to semantically reorganize and simplify redundant input paths, thereby generating the optimal topological sequence S_{tgt}. This paradigm is driven by large-scale data and encodes the shared geometric rules, curve fitting criteria, and node control logic across different fonts into a unified parametric representation, which yields strong cross-style generalization. The instruction template is shown in Tab. 6 #2.

G. VISION ENCODER

The difficulty of Chinese character vectorization lies in two aspects: restoring the visual appearance of characters at the pixel level and ensuring that the generated results follow valid topological structures in the vector representation. To this end, we construct a hybrid visual encoder for geometric information modeling. This encoding framework avoids the aspect ratio distortion caused by forced deformation and combines dynamic-resolution input, two-dimensional rotary positional encoding, and a hierarchical hybrid attention design to progressively build multi-level feature representations, from fine-grained stroke details to global semantics and topological structure.

G.1. Native-Resolution Visual Encoding.

Because the distribution of Bézier curve control points strongly depends on the original geometric features of the glyph, conventional resizing or padding at the input stage can easily alter the original aspect ratio and spatial organization, thereby weakening or even distorting the high-frequency geometric information. Therefore, we adopt an input strategy that supports native resolution. For any raster glyph image $I_{\text{src}} \in \mathbb{R}^{H\times W\times 3}$ with resolution $H \times W$, we divide it into a sequence of non-overlapping patches of fixed size $P \times P$. The patch grid has dimensions $N_h \times N_w$, where $N_h = \lceil H/P \rceil$ and $N_w = \lceil W/P \rceil$. The (i, j)-th patch $I_{i,j}$ is defined as:

$$I_{i,j} = I_{\text{src}}\big[(i-1)P + 1 : iP, ; (j-1)P + 1 : jP, ; :\big], \quad (21)$$

where $1 \le i \le N_h$ and $1 \le j \le N_w$. This strategy ensures that the model can adaptively encode glyphs with arbitrary aspect ratios while avoiding geometric information loss during preprocessing.

G.2. 2D Rotary Positional Encoding.

Under dynamic-resolution input, to more accurately represent the relative spatial relations among the components of a Chinese character in the two-dimensional plane, we embed two-dimensional rotary positional encoding (2D-RoPE) into the gridded feature representation. For any query vector q and key vector k in the feature space, we split the feature dimension D into two equal subspaces and apply positional encoding according to the row coordinate i and the column coordinate j, respectively. This decoupled design injects positional information into the representations along the two directions. The corresponding rotation transformation is defined as:

$$f_{2\text{D}}(x, i, j) = \begin{pmatrix} \mathcal{R}_i^{\Theta} & 0 \\ 0 & \mathcal{R}_j^{\Theta} \end{pmatrix} x, \quad (22)$$

where $\mathcal{R}_i^{\Theta}$ and $\mathcal{R}_j^{\Theta}$ are block-diagonal rotation matrices parameterized by the frequency set Θ. Crucially, the inner product between features depends only on their relative distance on the grid:

$$\langle f_{2\text{D}}(q, i, j), f_{2\text{D}}(k, i', j')\rangle = q^{\top}\mathcal{R}_{i-i',j-j'}k. \quad (23)$$

This translation invariance enables the model to directly capture relative geometric structure defined by the physical stride P, thereby preserving stable spatial correspondences across different resolutions and aspect ratios.

G.3. Multi-Scale Window-based Hybrid Attention.

To preserve high-resolution features while maintaining computational efficiency, we design a window-based hybrid attention mechanism. The visual encoder contains L Transformer blocks and adopts a hierarchical window attention strategy:

- **Progressive Local Window:** In the shallow layers, we adopt a hierarchical small-window strategy, where the window sizes are set to $(w_1, w_2, w_3) = (2, 4, 8)$ and applied cyclically. Combined with the shifted window mechanism, the model strengthens its ability to capture stroke details within a progressively expanding receptive field.
- **Global-Local Interleaving:** In the deep layers, we enlarge the window size to 16×16 to cover most Chinese character components. In particular, we insert global attention in specific layers, where self-attention is computed over the full sequence to model long-range dependencies across components.

G.4. Spatial Visual Token Aggregation.

To match the input dimension constraints of the large language model (LLM) and reduce the sequence length, we introduce a spatial aggregation module at the end of the visual encoder. We reshape the visual feature sequence $Z \in \mathbb{R}^{(N_h\cdot N_w)\times C}$ into a two-dimensional grid $\mathcal{F} \in \mathbb{R}^{N_h\times N_w\times C}$, and perform depth-wise concatenation over each adjacent 2×2 neighborhood:

$$v_{i,j} = [\mathcal{F}_{2i,2j} \oplus \mathcal{F}_{2i+1,2j} \oplus \mathcal{F}_{2i,2j+1} \oplus \mathcal{F}_{2i+1,2j+1}]. \quad (24)$$

We then use a multilayer perceptron (MLP) to project the aggregated features into the LLM embedding space D_{LLM}:

$$v'_{i,j} = W_2(\sigma(W_1 v_{i,j} + b_1)) + b_2, \quad (25)$$

this process reduces the visual sequence length to $\frac{N_h N_w}{4}$, significantly lowering the computational complexity while preserving dense semantic integrity through channel expansion.

Table 7. Detailed configuration of the Vision Encoder.

Configuration	Value / Setting
Model Capacity	
Hidden Size	1280
Intermediate Size	3456
Number of Heads	16
Total Transformer Blocks	16
Input & Global Settings	
Input Resolution Strategy	Dynamic (Native Aspect Ratio)
Patching Strategy	Non-overlapping $P \times P$ Grid
Positional Embedding	2D-RoPE
Shallow Layers (Blocks 1 – 6): Local Geometry	
Attention Mechanism	Window-based Self-Attention
Window Strategy	Shifted Window (Cyclic)
Window Sizes (w_1, w_2, w_3)	$\{2, 4, 8\}$
Deep Layers (Blocks 7 – 16): Global Structure	
Attention Mechanism	Large Window & Global Attention
Window Size	16×16
Global Attention Indices	Blocks $\{7, 10, 13, 16\}$
Visual Token Aggregation	
Spatial Aggregation	2×2 Neighbor Concatenation
Projection Layer	2-layer MLP
Sequence Reduction Rate	$4\times$ $(\frac{N_h N_w}{4})$
In Channel	1280
Out Channel	1536

H. LANGUAGE MODELING

We adopt the lightweight code model Qwen-2.5-Coder-7B as the core decoder to learn an end-to-end mapping from multimodal reference cues to normalized SVG XML sequence generation. In the few-shot reference setting, the

input sequence consists of the character to be optimized and its corresponding reference set. Specifically, the input stream contains the visual–code pair $(I_{\text{src}}, \mathbf{Z}_{\text{src}})$ of the target character, together with a reference set of N samples, $\mathcal{R} = \{(I_n^{\text{ref}}, \mathbf{Z}_n^{\text{ref}})\}_{n=1}^{N}$, where $\mathbf{Z}$ denotes the SVG XML token sequence of the corresponding character. The model autoregressively parameterizes the conditional generation probability of the target sequence S_{tgt} as follows:

$$p(S_{\text{tgt}} \mid I_{\text{src}}, \mathbf{Z}_{\text{src}}, \mathcal{R}) = \prod_{i=1}^{L} p(c_i \mid c_{<i}, I_{\text{src}}, \mathbf{Z}_{\text{src}}, \mathcal{R}). \tag{26}$$

To strengthen the model's ability to capture fine-grained geometric features and mitigate the attenuation of multimodal semantics in deep autoregressive decoding, we introduce a multi-layer visual injection mechanism. This design builds a hierarchical visual feature injection architecture from the visual encoder to the language decoder, enabling deep coupling of multi-scale features in the latent space.

H.1. Multi-Scale Feature Projection

We extract intermediate features $F^{(k)}$ from three representative levels of the dual-branch visual encoder, with indices $k \in \{1, 2, 3\}$. We then use a vision–language fusion module $g^{(k)}(\cdot)$, implemented as a two-layer MLP, to perform spatial downsampling and dimensional projection, aligning these features precisely with the hidden space of the language model:

$$V^{(k)} = g^{(k)}(F^{(k)}) \in \mathbb{R}^{N_v \times D_{\text{llm}}}, \quad k \in \{1, 2, 3\}, \tag{27}$$

where, $V^{(k)}$ encodes progressive semantic priors, ranging from local contour details to global topological structure.

H.2. Residual Layer Injection

Let the decoder contain L_{dec} Transformer layers, and let the input hidden state of the l-th layer be $H^{(l)} \in \mathbb{R}^{T \times D_{\text{llm}}}$, where T denotes the total sequence length after concatenating the reference set, the sample to be optimized, and their textual instruction. Let $\mathcal{I}_{\text{vis}}$ denote the index set of all visual positions in the sequence.

We predefine a set of shallow decoder layers, $\mathcal{L}_{\text{inj}} = \{l_1, l_2, l_3\}$, as information injection points. At these specific layers, we inject the multi-scale visual features $V^{(k)}$ into the hidden states at the corresponding visual positions through a residual formulation:

$$\tilde{H}^{(l_k)}_{\mathcal{I}_{\text{vis}}} = H^{(l_k)}_{\mathcal{I}_{\text{vis}}} + V^{(k)}, \tag{28}$$

$$H^{(l_k+1)} = \mathcal{B}^{(l_k)}(\tilde{H}^{(l_k)}), \tag{29}$$

where, $\mathcal{B}^{(l)}(\cdot)$ denotes the l-th Transformer decoder block. This cross-modal residual connection forces the model to directly trace back to the geometric cues of the original glyph and the topological patterns of the reference templates at every stage of generation. Experiments show that this mechanism effectively guides the model to maintain high topological accuracy in complex Bézier path reconstruction, thereby elevating the vectorization task from low-level pixel fitting to semantic-level intent reconstruction.

Table 8. Detailed configuration of the language modeling network (Qwen2.5-Coder-7B).

Configuration	Value/Setting
Model Type / Architecture	Decoder-only Causal LM (`Qwen2ForCausalLM`)
Parameters	7.61B (Non-Embedding: 7.07B)
Transformer Layers (L)	28
Hidden Size (d_{model})	3584
Attention Heads (n_h)	28
KV Heads (n_{kv}, GQA)	4
Head Dim ($d_h = d_{\text{model}}/n_h$)	128
FFN Intermediate Size (d_{ffn})	18928
Activation	SiLU (`silu`)
Normalization	RMSNorm ($\epsilon = 10^{-6}$)
Positional Encoding	RoPE
RoPE Base (θ)	1,000,000
Max Context Length	131,072
Vocab Size (config.json)	152,064
Vocab Size (tech report)	151,646
Tie Word Embeddings	false
Attention Dropout	0.0
Sliding Window Size	131,072
Use Sliding Window	false
Max Window Layers	28
dtype	bfloat16 (`bf16`)
BOS / EOS Token ID	151643 / 151643

I. TASKS DEFINITION

VFRefine is dedicated to the following two core tasks:
Vector Glyph Refinement: This task serves as the primary objective of this study and is mainly used to assess the model's ability to convert geometrically redundant vector glyphs into compact, industry-compliant SVG representations. We evaluate on the VFRefine-1.2M test set, where the input consists of a raster image and non-canonical SVG code. Under this setting, the model not only needs to preserve consistency between the output glyph and the original visual appearance, but also needs to optimize the topology of the input Bézier curves and sparsify their control points based on an understanding of the glyph design intent, thereby producing a geometrically optimal vector representation.
Image-to-Vector Generation: Benefiting from the cross-modal semantic alignment established during training, VFRefine inherently has the ability to map directly from pixel space to vector parameter space. This task mainly evaluates how well the model reconstructs high-precision vector glyphs solely from input raster font images in the absence of an initial SVG contour prompt. This evaluation further shows that the model not only handles the normalization and compression of existing vector glyphs, but also reflects a strong internal representation of Chinese character geometry, indicating its potential as an end-to-end vectorization method.

J. BASELINES

To systematically evaluate the performance of VFRefine, we compare it with two representative technical paradigms: traditional graphics algorithms based on geometric rules and general-purpose multimodal large language models. For the graphics methods, we select representative tools from both the open-source community and industry. For the MLLM baselines, to ensure a fair comparison, we conduct supervised full-parameter fine-tuning on the VFRefine-1.2M dataset. The compared models are as follows:

- **Potrace [27]:** It is a widely used bitmap vectorization method in the open-source community. Its core idea is to trace polygons along bitmap boundaries and then fit Bézier curves to convert discrete contours into vector representations. As a typical geometry rule-driven method, this algorithm offers some advantage in computational cost, but because it mainly relies on contour-based geometric approximation, its ability to represent glyph semantic structure remains relatively limited.
- **Adobe Image Trace (AIT):** It is an industrial-grade image vectorization tool integrated into Adobe Illustrator. Its processing pipeline is based on iterative error optimization and contour smoothing, enabling it to produce geometric boundaries with relatively high precision. In terms of methodological characteristics, this tool is a representative non-learning-based vectorization system and reflects the strong performance of such methods on geometric fitting tasks.
- **StarVector-8B [25]:** It is a multimodal large language model that shows strong performance in visual semantic understanding and high-quality vector graphic (SVG) code generation. Built on the StarCoder2-7B architecture and optimized on the two-million-scale SVG-Stack dataset, it has strong visual feature alignment and native SVG primitive inference capabilities. It can accurately reconstruct complex structured charts and text, making it suitable for inverse rendering and multimodal code generation tasks that require compact, precise, and semantically faithful outputs.
- **InternSVG-8B [36]:** It is a multimodal large language model that shows strong performance in unified SVG understanding, editing, and generation. The model is built on the InternVL3-8B architecture, which combines the InternViT-300M visual encoder with the Qwen2.5-7B language model, and is optimized on the SAgoge multimodal dataset with more than 16 million samples. It introduces SVG-specific special tokens and a subword-based embedding initialization strategy. The model has strong capability in handling static long-sequence illustrations, complex scientific charts, and dynamic animations. Through unified modeling, it enables positive transfer across tasks, making it suitable for general SVG visual perception and digital graphic creation tasks that require strong generalization and deep hierarchical structure inference.
- **DeepSeek-VL-7B [19]:** It is a multimodal large language model that shows strong performance in logical reasoning and fine-grained visual processing. Optimized on large-scale chain-of-thought (CoT) data, the model can handle complex structured information and infer implicit topological logic, making it suitable for sequence generation tasks that require high-precision reasoning.
- **Qwen2.5-VL-3B [2]:** It is a lightweight and efficient model with a native dynamic resolution mechanism (Naive Dynamic Resolution). The architecture can process image inputs with arbitrary aspect ratios and shows strong performance in optical character recognition (OCR) and fine-grained visual recognition tasks. It can effectively capture subtle geometric features.
- **InternVL3-8B [49]:** It is an open-source multimodal large model built on a "ViT + MLP + LLM" architecture. Through native multimodal pretraining, it jointly learns visual and linguistic capabilities, which substantially improves understanding and reasoning. Compared with the previous InternVL2.5, it shows stronger performance in native multimodal pretraining, long-context understanding, and complex tasks involving tool use, GUI, video, and industrial scenarios.
- **Qwen3-VL-4B [1]:** It is the latest iteration of the Qwen-VL series and substantially strengthens visual reasoning and long-context understanding. The model retains and improves the dynamic-resolution input mechanism, which preserves high-frequency geometric features in images, making it a strong baseline among current general-purpose MLLMs for visual generation tasks.

Implementation Details for Baselines. To ensure a consistent comparison across methods, all MLLM baselines are instruction-tuned on the VFRefine-1.2M training set. For the graphics-based methods, Potrace and AIT, we uniformly enable high-precision settings to fully exploit their geometric contour fitting capability.

K. EVALUATION METRICS

Chinese vector glyph optimization needs to balance two competing objectives. First, under rasterized rendering, the optimized result should match the visual appearance of the original glyph as closely as possible. Second, while preserving glyph quality, it should minimize the number of Bézier curve control points. Based on these two evaluation dimensions, we design a multi-level evaluation framework for VFRefine and conduct a comprehensive analysis from three perspectives: visual fidelity, contour structure consistency, and optimization gain. At the visual level, we use raster-based metrics, including IoU and MSE. At the structural level, we use geometric contour metrics, namely CD and DTW. In addition, we introduce Reduction Ratio as a statistical metric to characterize the optimization efficiency brought by control-point compression.

K.1. Intersection over Union

Intersection over Union (IoU) is the most commonly used metric for measuring shape overlap. In our task, it quantifies the consistency of the optimized vector glyph with the target canonical glyph in terms of visual region. We render the generated SVG code S_{des} and the target ground truth S_{gt} into high-resolution binary images I_{des} and I_{gt}. IoU is computed as the ratio between the intersection area and the union area of the two images:

$$\text{IoU} = \frac{\text{Area}(I_{\text{des}} \cap I_{\text{gt}})}{\text{Area}(I_{\text{des}} \cup I_{\text{gt}})}. \tag{30}$$

A higher IoU indicates that VFRefine better preserves the overall glyph configuration and stroke-width characteristics of the target glyph during generation. Even after control-point compression, its rasterized output remains highly consistent with the original visual form. This result shows that the method achieves control-point sparsification while effectively avoiding obvious shape degradation and structural distortion.

K.2. Mean Squared Error

Mean Squared Error (MSE) [39] provides a finer-grained pixel-level measure than IoU. It computes the mean squared difference in pixel intensity between the generated glyph image and the ground-truth image. This metric is particularly sensitive to anti-aliasing details along edges and small positional shifts. The formula is given as follows:

$$\mathrm{MSE} = \frac{1}{H \cdot W} \sum_{i=1}^{H} \sum_{j=1}^{W} \left(I_{\mathrm{des}}(i,j) - I_{\mathrm{gt}}(i,j)\right)^2 . \tag{31}$$

In the vector optimization task, a low MSE indicates that the reconstructed glyph image remains highly consistent with the target ground truth at the pixel level.

K.3. Chamfer Distance

Chamfer Distance (CD) [6] is used to measure the geometric similarity between two point sets. It samples point sets P_{des} and P_{gt} from the two contour curves and computes the average distance from each point to its nearest neighbor in the other set, thereby evaluating contour alignment independently of point density.

$$\mathrm{CD}(P_{\mathrm{des}}, P_{\mathrm{gt}}) = \frac{1}{|P_{\mathrm{des}}|} \sum_{x \in P_{\mathrm{des}}} \min_{y \in P_{\mathrm{gt}}} \|x-y\|_2 + \frac{1}{|P_{\mathrm{gt}}|} \sum_{y \in P_{\mathrm{gt}}} \min_{x \in P_{\mathrm{des}}} \|y-x\|_2 . \tag{32}$$

CD mainly measures shape approximation at the geometric level. In our evaluation setting, a smaller CD indicates higher spatial trajectory consistency between the generated vector path and the expert-constructed reference path. This metric focuses on the geometric alignment of the contour itself rather than the number of control points or their specific distribution.

K.4. Dynamic Time Warping

The essence of a Chinese character glyph is a vector path composed of Bézier curves, and defect annotation is also defined as modifications to these paths. Dynamic Time Warping (DTW) [26] is a sequence matching algorithm that computes the global path similarity between two coordinate sequences of possibly different lengths, $P = \{p_1, \ldots, p_n\}$ and $Q = \{q_1, \ldots, q_m\}$. DTW first uses dynamic programming to find an optimal warping path $\mathcal{W}^* = \{(i_k, j_k)\}_{k=1}^{K}$ with the minimum cumulative distance, where K is the length of the warping path. The cumulative distance $D(n, m)$ is obtained by the following recurrence:

$$D(i,j) = d(p_i, q_j) + \min \begin{cases} D(i-1, j) \\ D(i, j-1) \\ D(i-1, j-1) \end{cases} , \tag{33}$$

where, $d(p_i, q_j) = \|p_i - q_j\|_2$ denotes the local distance. To remove the effect of path length differences, we use the average per-step cost as the final evaluation metric:

$$\mathrm{DTW} = \frac{D(n,m)}{|\mathcal{W}^*|}. \tag{34}$$

DTW measures the discrepancy between the SVG coordinate sequence generated by the model and the ground-truth annotated sequence in terms of overall organization and sequence correspondence. Under this metric, a smaller value indicates that the MLLM models the global path structure more accurately and that the generated sequence is more consistent with the ground-truth sequence in the ordering and progression process, thereby reflecting stronger SVG path understanding and generation ability.

K.5. Reduction Ratio

Reduction Ratio (RR) is the core metric for evaluating the optimization efficiency of VFRefine, and it quantifies the model's ability to remove geometric redundancy. We treat glyph optimization as a lossy data compression process, in which the visual appearance remains unchanged while the data size is reduced. This metric is defined as the relative reduction in the number of drawing commands N_{cmd} between the non-canonical input glyph S_{src} and the optimized output glyph S_{des}:

$$\mathrm{RR} = \frac{N_{cmd}(S_{\mathrm{src}}) - N_{cmd}(S_{\mathrm{des}})}{N_{cmd}(S_{\mathrm{src}})} \times 100\%. \tag{35}$$

A higher RR indicates that the model compresses redundant control points more effectively and removes invalid noisy paths while preserving the main glyph structure. Therefore, this metric directly reflects the improvement in the compactness of the generated vector code and further demonstrates its advantages in storage cost control and rendering efficiency.

L. QUANTITATIVE EVALUATION

L.1. Ablation on Refinement Paradigms

To analyze the effect of different inference paradigms on vectorization results, we divide the experiments into two settings, Few-Shot Reference-Guided and Zero-Shot Universal, and report a fine-grained comparison in Tab. 9. Under the Few-Shot Reference-Guided setting, the model exploits the style information and structural cues provided by reference samples to identify and reconstruct the topological relations among complex strokes more effectively. With this reference-guided mechanism, VFRefine aligns the geometric features of the target glyph and the template more accurately, and thus achieves the best results on both DTW, which reflects topological consistency, and CD, which measures geometric approximation quality. By contrast, the Zero-Shot Universal setting provides no explicit reference information and therefore places higher demands on the model's prior knowledge and generalization ability. Even under this setting, VFRefine still relies on universal design knowledge learned from large-scale data to maintain good generation stability and cross-style adaptability. Although its performance on accuracy-related metrics is slightly lower than that under the few-shot reference-guided setting, VFRefine still outperforms other baseline methods on both IoU and RR, which indicates strong overall competitiveness in reference-free scenarios.

L.2. Generalization on Character Visibility

To examine whether the capability of VFRefine is limited to directly memorizing character structures during training,

Table 9. Detailed quantitative comparison under different refinement paradigms.

Method	Few-Shot Reference-Guided					Zero-Shot Universal				
	IoU ↑	MSE ↓	CD ↓	DTW ↓	RR (%) ↑	IoU ↑	MSE ↓	CD ↓	DTW ↓	RR (%) ↑
StarVector-8B	0.788	0.070	0.1460	3.545	64.16	0.765	0.075	0.1582	3.828	63.85
InternSVG-8B	0.816	0.066	0.1280	3.036	66.87	0.795	0.072	0.1355	3.253	66.49
DeepSeek-VL-7B	0.836	0.053	0.0922	2.724	68.07	0.820	0.058	0.1056	2.957	67.81
Qwen2.5-VL-3B	0.883	0.039	0.0780	2.367	69.00	0.865	0.045	0.0890	2.582	68.75
InternVL3-8B	0.894	0.033	0.0612	1.921	70.43	0.878	0.039	0.0722	2.159	69.98
Qwen3-VL-8B	0.926	0.026	0.0420	1.559	72.26	0.912	0.031	0.0519	1.783	71.81
VFRefine (Ours)	**0.975**	**0.010**	**0.0122**	**0.948**	**76.85**	**0.962**	**0.014**	**0.0155**	**1.120**	**76.10**

we further divide the test set into Seen Characters and Unseen Characters and conduct comparative experiments accordingly. The results are reported in Tab. 10. The results show that VFRefine performs well on the Seen Characters subset, which indicates that when a character structure appears during training, the model not only maintains stable structural modeling ability but also completes effective style adaptation and geometric reconstruction for unseen font styles based on its existing topological understanding.

On the more challenging Unseen Characters* subset, the model no longer has direct structural experience for specific characters, yet VFRefine still shows good stability and adaptability. This result suggests that the model performance does not rely on superficial reproduction of training samples, but instead arises from more generalizable learning of Chinese character stroke organization patterns and Bézier curve construction mechanisms. In other words, VFRefine acquires transferable structural reasoning ability rather than simple memorization of specific character instances, and therefore maintains good performance in open-set character scenarios.

L.3. Comparison with Closed-Source Large Models

To further evaluate the competitiveness of VFRefine against stronger baselines, we introduce four representative closed-source large models, GPT-4o, Gemini-3.1-Pro, GPT-5.4, and Claude-Sonnet-4.6, for comparison. We divide the test set by font style into two subsets, Standard Typefaces and Decorative Typefaces, and evaluate all methods under the Zero-Shot Universal setting. The results are reported in Tab. 11.

Benefiting from large-scale training corpora and higher model capacity, closed-source large models generally outperform some open-source baselines on vector glyph generation. However, their performance gains remain limited, and the differences among the four closed-source models are also marginal. This observation suggests that general-purpose large models are approaching their capability limit on the highly specialized task of vector glyph refinement, and that scaling model size alone is unlikely to produce substantial quality improvements. Specifically, on the Standard Typefaces subset, the CD values of the four closed-source models fall within 0.0520–0.0578, while their DTW values lie within 2.352–2.583, which indicates only negligible differences among them.

In contrast, VFRefine outperforms the above closed-source models on all metrics across both subsets. On the Standard Typefaces subset, VFRefine reduces CD by 77.9% and DTW by 52.4% relative to GPT-5.4, the strongest closed-source model, while improving IoU by 8.7 percentage points. On the more challenging Decorative Typefaces subset, the performance of the closed-source models degrades further and remains similar across models, whereas VFRefine still maintains an advantage, achieving 0.937 IoU and 72.60% RR. These results indicate that, although general closed-source foundation models possess broad vision-language understanding capabilities, their performance remains inferior to that of VFRefine on the specialized task of vector glyph refinement, which requires precise path modeling and stroke topology reasoning. This finding further validates the effectiveness of our method in professional scenarios.

M. QUALITATIVE EVALUATION

To systematically evaluate the vectorization refinement effect of VFRefine, we select representative graphics-based methods and mainstream multimodal large language models for comparison. The outputs of all methods are shown in Fig. 12. The comparison shows that different technical routes exhibit distinct limitations on the vector glyph generation task.

Among traditional graphics tools, Potrace and AIT produce relatively smooth glyph contours, but their core mechanism relies on local curve fitting along raster image edges at the pixel level and lacks semantic awareness of the overall structure of Chinese character strokes. As a result, the resulting vector paths contain a large number of dense Bézier curve segments and control points, which not only increases file size but also forces designers to spend substantial effort on node cleanup and path adjustment in subsequent editing. This limitation makes these methods difficult to integrate into the industrial workflow of professional font production. For multimodal large language models, although instruction-tuned general models show an initial ability to convert raster inputs into vector code outputs, their geometric precision remains insufficient. The generated glyph contours exhibit clear coordinate deviations and local oscillations, and the stroke boundaries lack smoothness. Moreover, when handling the complex internal topological relations of Chinese characters, these models have limited modeling ability, which often leads to stroke adhesion and missing local structures. They therefore still fail to satisfy the requirements of font design for glyph regularity and consistency.

VFRefine provides a unified solution to the limitations of these two technical routes. By effectively modeling multimodal structural prior knowledge, it generates glyphs whose

Table 10. Quantitative comparison on Seen vs. Unseen characters.

Method	Seen Characters					Unseen Characters				
	IoU ↑	MSE ↓	CD ↓	DTW ↓	RR (%) ↑	IoU ↑	MSE ↓	CD ↓	DTW ↓	RR (%) ↑
StarVector-8B	0.826	0.068	0.1295	3.308	65.13	0.755	0.072	0.1642	3.782	63.26
InternSVG-8B	0.845	0.062	0.1186	2.912	67.12	0.782	0.068	0.1351	3.141	66.85
DeepSeek-VL-7B	0.860	0.046	0.0822	2.539	68.18	0.849	0.062	0.1063	2.967	67.93
Qwen2.5-VL-3B	0.893	0.030	0.0680	2.011	69.74	0.870	0.046	0.0849	2.765	68.20
InternVL3-8B	0.902	0.025	0.0572	1.690	71.65	0.886	0.043	0.0651	2.112	69.04
Qwen3-VL-8B	0.938	0.018	0.0367	1.362	72.53	0.911	0.030	0.0472	1.773	71.99
VFRefine (Ours)	**0.982**	**0.008**	**0.0105**	**0.885**	**77.50**	**0.960**	**0.015**	**0.0146**	**1.013**	**76.13**

Table 11. Quantitative comparison with closed-source large models on standard and decorative typefaces.

Method	Standard Typefaces					Decorative Typefaces				
	IoU ↑	MSE ↓	CD ↓	DTW ↓	RR (%) ↑	IoU ↑	MSE ↓	CD ↓	DTW ↓	RR (%) ↑
GPT-4o	0.863	0.048	0.0578	2.583	67.42	0.841	0.056	0.0694	2.917	65.80
Gemini-3.1-Pro	0.858	0.050	0.0563	2.512	67.85	0.836	0.058	0.0672	2.845	66.13
GPT-5.4	0.871	0.045	0.0520	2.352	68.36	0.849	0.053	0.0638	2.763	66.72
Claude-Sonnet-4.6	0.867	0.046	0.0541	2.468	68.03	0.845	0.054	0.0655	2.810	66.45
VFRefine (Ours)	**0.958**	**0.016**	**0.0115**	**1.120**	**75.12**	**0.937**	**0.022**	**0.0129**	**1.241**	**72.60**

visual fidelity remains highly consistent with the target ground truth and accurately reproduces the morphological details of each stroke. In path representation, VFRefine substantially reduces the number of Bézier curve control points and greatly decreases geometric redundancy relative to graphics-based methods such as Potrace. In curve quality, compared with the outputs of general multimodal large language models, the paths generated by VFRefine achieve both good smoothness and structural regularity, while remaining highly editable, which helps improve the productivity of font designers.

N. HUMAN EVALUATION

Since VFRefine targets real-world font design workflows, automated metrics alone are insufficient to fully assess its output quality. We therefore design a blind study and invite 20 designers with more than five years of font design experience to participate in the evaluation. The study examines two aspects: the visual consistency between the generated glyphs and the target glyphs, and the practical usability of the generated results in industrial production, in order to assess whether the model outputs satisfy professional delivery requirements.

N.1. Visual Indistinguishability Test

Whether generated glyphs are visually indistinguishable from the target glyphs refined by experts is a key criterion for evaluating vectorization quality. We verify this from both objective measurement and subjective perception.

For objective evaluation, we compute MSE and IoU on the rasterized outputs of all methods over the test set. The results are reported in Tab. 12. Graphics-based methods represented by AIT achieve relatively strong IoU because they directly fit pixel-level edge information. By contrast, the multimodal large language model baselines are constrained by coordinate drift and therefore yield consistently lower IoU. Under a highly compact path topology, VFRefine still attains visual accuracy comparable to that of graphics-based methods, which indicates a good balance between structural simplicity and geometric fidelity. For subjective evaluation, we design a visual Turing test for all compared methods. Specifically, we randomly sample 50 generated results from each method, render them as images, and present each rendered image together with its corresponding Ground Truth in a random left-right arrangement. The designers are then asked, without time limits, to identify which one is the ground truth. The rationale is as follows: if the identification accuracy is significantly above chance, the generated results contain perceptible visual defects; otherwise, the closer the accuracy is to 50%, the smaller the visual difference between the generated glyphs and the ground truth, and the closer the two are to being visually indistinguishable.

Table 12. Comparison of raster-based visual metrics (MSE, IoU) and human visual distinguishability scores across all methods. "Human Distinguishability" denotes the accuracy of experts in identifying the generated glyph against the Ground Truth (closer to 50% indicates better consistency).

Method	MSE ↓	IoU ↑	Human Distinguishability (Target: 50%)
Potrace	0.009	0.977	54.6%
Adobe Image Trace (AIT)	**0.004**	**0.986**	**51.4%**
StarVector-8B	0.068	0.824	98.6%
InternSVG-8B	0.058	0.845	97.1%
DeepSeek-VL-7B	0.046	0.869	93.7%
Qwen2.5-VL-3B	0.039	0.893	89.5%
InternVL3-8B	0.033	0.904	85.4%
Qwen3-VL-8B	0.024	0.938	75.9%
VFRefine (Ours)	0.010	0.975	53.1%

Fig. 12. Qualitative comparison with state-of-the-art methods on the VFRefine-1.2M test data.

The user study results are reported in the fourth column of Tab. 12. For the baselines built on general multimodal large language models, the identification accuracy mostly exceeds 80%, which indicates that designers can quickly distinguish generated results from the ground truth through visual cues such as boundary jitter and structural deformation. This further reflects the limited geometric precision of such methods. Although the graphics-based methods are difficult to distinguish by eye, with Potrace at 54.6% and AIT at 51.4%, the topology evaluation in the previous section shows that this comes at the cost of substantial node redundancy. Our method, VFRefine, achieves an identification accuracy of 53.1%, which is very close to random guessing. This result indicates that, after rasterized rendering, the visual appearance of its generated vector glyphs is difficult to distinguish from expert-refined results, while this effect is not obtained by relying on redundant path complexity.

N.2. Topological Editability Evaluation

High visual consistency is a necessary condition for evaluating vector glyph quality. However, in practical font production, the key factor that determines designer efficiency and workflow feasibility is the topological quality of the vector paths, namely the node distribution and handle logic of the Bézier curves. We therefore further conduct a systematic comparative analysis of the vector path topology generated by all baseline methods.

Scoring Standards. We adopt a scoring scheme based on a 5-point Likert scale. To ensure objective evaluation, we define detailed scoring criteria for the three dimensions: Node Economy (NE), Control Logic (CL), and Production Readiness (PR). The full rubric is reported in Tab. 13.

To visualize the topological differences across methods, we present the vector contours generated for the same character by eight baseline methods and VFRefine side by side. All samples are anonymized and randomly ordered to avoid prior bias. Designers score the topological quality of each method from multiple dimensions, and the results are summarized in Tab. 14.

From the score distribution, graphics-based methods such as Potrace and AIT receive low scores across all dimensions, although their rasterized visual deviations are small. The designers provide consistent explanations: these tools place a large number of control nodes at non-extremal positions, and their paths contain obvious redundant short segments. Together, these defects lead to high post-editing cost and limit their usability in production workflows.

Among the methods based on large language models, the overall user scores increase as the capability of the underlying model improves. However, the designer feedback also reveals an inherent limitation of such methods. Although general multimodal large language models tend to generate paths with fewer nodes, their outputs frequently contain structural hallucinations and large coordinate deviations, which cannot satisfy commercial delivery requirements without substantial manual correction. Compared with all baselines, VFRefine achieves clearly higher scores on every metric. The designers' comments focus on two aspects. First, the topology generated by VFRefine is more reasonable in both the number and placement of nodes. Second, the handle directions strictly follow the professional font design convention of drawing at extremal points, which makes the results directly suitable for production workflows.

Beyond the subjective scores, we further introduce a quantitative test of operational efficiency, and report the results in the rightmost column of Tab. 14. Designers are asked to revise the outputs of each method to a commercial

Table 13. Detailed grading rubric for the 5-point Likert Scale used in the expert evaluation. The criteria cover three dimensions: Node Economy (NE), Control Logic (CL), and Production Readiness (PR).

Score	Node Economy (NE)	Control Logic (CL)	Production Readiness (PR)
1	**Extreme Redundancy.** Nodes are excessively dense and overlapping; simple shapes are composed of hundreds or thousands of minute line segments.	**Chaotic.** Broken handles; sharp corners appear within curves; cross-intersecting handles exhibit a total lack of geometric logic.	**Unusable.** Requires complete reconstruction; repairing these paths is more labor-intensive than redrawing from scratch.
2	**High Redundancy.** Superfluous nodes appear on straight segments; dense node clusters form at corners.	**Poor Logic.** Handles deviate from axes; curves exhibit only positional continuity (G0); difficult to edit.	**Major Repairs.** Obvious visual artifacts present; requires extensive manual cleanup (repair time $>$ 2 min/glyph).
3	**Moderate Redundancy.** Node count exceeds the necessary minimum, though distribution is relatively uniform.	**Acceptable.** Curves are smooth (G1), but handles are not positioned at extrema; handle orientation is arbitrary.	**Moderate Repairs.** Visually acceptable, yet topology is disordered; requires routine optimization.
4	**Good Economy.** Absence of redundant nodes on straight segments; minimal redundancy on curved segments.	**Standard Logic.** Handles are predominantly horizontal or vertical; exhibits good curvature continuity (G2).	**Minor Tweaks.** Robust structure; requires only fine-tuning of specific details.
5	**Minimal/Optimal.** Achieves the theoretical minimum node count (based on extremum point plotting); zero redundancy.	**Expert Logic.** Handles at extrema are strictly orthogonal; exhibits perfect smoothness and symmetry.	**Commercial Ready.** Directly exportable as a font file; fully compliant with foundry delivery standards.

Table 14. Human evaluation results on topological editability and production efficiency. We report Likert scores (1-5, higher is better) for topological quality and the average time required for designers to fix the result to a commercial standard (seconds, lower is better).

Method	Likert Scale (1-5) ↑			Efficiency ↓
	Node Economy	Control Logic	Prod. Readiness	**Fix Time (s)**
Potrace	2.04 ± 0.5	1.67 ± 0.5	1.58 ± 0.3	96.2 ± 12.7
Adobe Image Trace (AIT)	2.57 ± 0.4	2.27 ± 0.5	2.39 ± 0.3	85.2 ± 9.6
StarVector-8B	2.76 ± 0.6	2.36 ± 0.6	2.50 ± 0.6	74.5 ± 9.7
InternSVG-8B	2.83 ± 0.7	2.40 ± 0.5	2.66 ± 0.5	72.6 ± 8.4
DeepSeek-VL-7B	3.42 ± 0.6	2.82 ± 0.6	3.25 ± 0.5	58.8 ± 8.0
Qwen2.5-VL-3B	3.04 ± 0.6	2.50 ± 0.4	2.87 ± 0.5	66.7 ± 7.6
InternVL3-8B	3.30 ± 0.7	2.68 ± 0.6	3.10 ± 0.7	59.5 ± 6.8
Qwen3-VL-8B	3.86 ± 0.4	3.53 ± 0.4	3.72 ± 0.4	41.7 ± 5.3
VFtRefine (Ours)	**4.65 ± 0.4**	**4.25 ± 0.4**	**4.38 ± 0.4**	**15.1 ± 2.8**
Ground Truth	*4.92 ± 0.3*	*4.95 ± 0.1*	*4.96 ± 0.1*	*0.0 ± 0.0*

standard and record the required time. VFRefine requires only 15.1 seconds on average, while Adobe Image Trace and the strongest baseline, Qwen3-VL, require 85.2 seconds and 41.7 seconds, respectively. This corresponds to about 5.6 times and 2.8 times higher efficiency for VFRefine. From the perspective of production cost, this result further shows that VFRefine not only produces more topologically compliant outputs than existing methods, but also effectively reduces the time cost of manual intervention in real industrial settings.

O. ABLATION STUDIES

O.1. Impact of Dual-Modal Reference Guidance

Under the few-shot reference-guided setting, the core idea of VFRefine is to introduce the SVG code of the reference samples into the model input as an explicit topological prompt, which provides direct geometric priors for learning Bézier curve sparsification. To verify whether the introduction of the SVG modality is necessary for the model to learn an effective sparsification strategy, we construct an ablation study in which only the input modality of the reference samples is controlled while keeping the visual encoder and language model architecture unchanged. The experiment includes two configurations. In the **Visual-Only Reference** setting, the reference input contains only the raster image of the target character, I_{ref}, and the model must infer the geometric construction logic of the reference font entirely from pixel-level visual information. In the **Dual-Modal Reference (Ours)** setting, the reference input includes both the raster image I_{ref} and the corresponding normalized SVG code Z_{ref}, which provides precise control-point coordinates and path-structure information.

Tab. 15 summarizes the results under the two configurations. Under the Visual-Only setting, all metrics drop noticeably, which indicates that raster pixel information alone does not provide sufficient geometric constraints for learning sparse control-point layouts. Without explicit path-structure input, the model tends to stack more control points to approximate the target contour, which leads to vector paths with substantial geometric redundancy. Under the Dual-Modal configuration, introducing SVG code improves the redundancy rate RR to 76.85% and raises IoU to 0.975, yielding

clear gains on both metrics. This comparison shows that the precise geometric parameters and topological constraints carried by SVG code compensate for the limited structural expressiveness of raster images. By leveraging the control-point distribution patterns in the reference SVG data, the model achieves effective Bézier curve sparsification and topological regularization while maintaining high visual fidelity.

Table 15. Ablation study on the impact of dual-modal reference guidance in Few-Shot mode. "Visual-Only" implies using only raster images as references, while "Dual-Modal" includes both images and SVG code.

Input Modality	Visual Metrics		Topological Metrics		
	IoU ↑	MSE ↓	CD ↓	DTW ↓	RR (%) ↑
Visual-Only Reference	0.920	0.024	0.0218	1.307	68.12
Dual-Modal Reference	**0.975**	**0.010**	**0.0122**	**0.948**	**76.85**

O.2. Effectiveness of Multi-Scale Layer Injection

Mainstream multimodal large models usually fuse visual and textual information by input-side concatenation, where the feature sequence extracted by the visual encoder is prepended to the text embedding sequence and then fed into the language model. This scheme has a clear limitation. Because SVG code yields a long token sequence, the attention assigned by deep decoder layers to the visual prefix tokens gradually weakens as autoregressive generation proceeds, causing geometric constraint information to decay over long-range transmission. To address this issue, we design a hierarchical injection strategy that continuously injects multi-scale visual features into decoder layers at different depths through residual connections, thereby maintaining effective geometric constraints throughout generation.

To evaluate the effectiveness and necessity of this design, we construct a controlled experiment with two feature fusion schemes. In the **Input Concatenation (Baseline)** setting, we remove the residual injection modules inside the decoder and retain only the final-layer output of the visual encoder, which is linearly projected and prepended to the input sequence of the language model as the sole source of visual information. In the **Layer Injection** setting, we keep the full architecture described in Sec. H and inject the multi-scale features extracted from different levels of the visual encoder into the Transformer layers at the corresponding depths of the decoder.

Table 16. Ablation study on feature fusion strategies. We compare standard Input Concatenation against our Multi-Scale Layer Injection.

Fusion Strategy	Geometric Fidelity		Structural Topology		Long-Seq Drop ↓
	IoU ↑	MSE ↓	CD ↓	DTW ↓	
Input Concatenation	0.930	0.026	0.0241	1.433	8.6%
Layer Injection (Ours)	**0.965**	**0.011**	**0.0133**	**0.962**	**2.0%**

The quantitative comparison of the two schemes is reported in Tab. 16. Input Concatenation achieves an IoU of 0.930, which indicates that it still has reasonable contour coverage ability. However, on MSE and CD, which are more sensitive to geometric precision, it shows a clear gap from Layer Injection. The difference in DTW is particularly notable, since this metric directly reflects the topological alignment between the generated path sequence and the ground truth. The advantage of Layer Injection on this metric indicates that its generated paths are closer to the reference standard in both control-point arrangement and curve trajectory.

We further introduce Long-Seq Drop as an auxiliary metric, which measures the IoU degradation of long-sequence samples relative to short-sequence samples and thus reflects stability in long-range generation. The results show that Input Concatenation suffers an 8.6% performance drop on long-sequence generation tasks. This observation is consistent with the theoretical analysis. When visual information is injected only once at the input side, the geometric constraint signal is gradually diluted as decoding proceeds through deeper layers, which degrades the generation quality near the tail of long sequences. By continuously supplying visual cues at multiple decoder depths, Layer Injection keeps the performance drop under 2.0% in the long-sequence setting and effectively mitigates the attenuation of deep geometric constraints.

O.3. Ablation on Hierarchical SVG Decoding

The hierarchical SVG decoding strategy consists of three core designs: primitive-based staged command-parameter generation, command-conditioned parameter decoding, and the hierarchical consistency constraint $\mathcal{L}_{\text{hc}}$. To verify the independent contribution of each component and to examine whether the internal decoding constraints conflict with strong external reference guidance during optimization, we conduct a set of progressive ablation studies under two settings: Zero-Shot Universal and Few-Shot Reference-Guided. We use standard flattened autoregressive decoding as the baseline and introduce the above designs incrementally.

(A) Flat Autoregressive: Configuration (A) removes primitive-level decomposition, places command tokens and coordinate parameter tokens in the same flattened sequence, and trains the model end to end with a unified cross-entropy loss, corresponding to the standard autoregressive SVG generation paradigm. Configuration (B), **Primitive Decomposition**, introduces primitive-level sequence reorganization, parses the token stream into a staged generation order of command-parameter groups, and applies $\mathcal{L}_{\text{cmd}}$ and $\mathcal{L}_{\text{coord}}$ separately for independent supervision, while the parameter decoding head does not receive command embedding information. Configuration (C), **Command-Conditioned Decoding**, builds on (B) by concatenating the embedding of the current command token, $e(c_i)$, to the hidden state of parameter tokens, so that parameter prediction is conditioned on the command type. Configuration (D), **Full Hierarchical (Ours)**, further introduces the hierarchical consistency constraint $\mathcal{L}_{\text{hc}}$ on top of (C), using the primitive initial state as an anchor to preserve structural information during parameter generation.

The bimodal experimental results in Tab. 17 reveal the incremental contribution of hierarchical decoding and its coupling effect with external reference signals. In the zero-shot setting, the flattened autoregressive baseline (Configuration A) places command tokens and coordinate tokens in the same prediction space. The model thus lacks an explicit distinction between structural decision-making and geometric re-

Table 17. Ablation study on the Hierarchical SVG Decoding strategy under both Zero-Shot and Few-Shot settings. Components are progressively added to evaluate their impact and the coupling effect with external reference prompts.

Decoding Configuration	Zero-Shot (Universal)					Few-Shot (Reference-Guided)				
	IoU ↑	MSE ↓	CD ↓	DTW ↓	RR (%) ↑	IoU ↑	MSE ↓	CD ↓	DTW ↓	RR (%) ↑
(A) Flat Autoregressive	0.941	0.038	0.0218	1.620	72.10	0.945	0.025	0.0210	1.280	74.21
(B) + Primitive Decomposition	0.948	0.030	0.0231	1.412	73.63	0.955	0.020	0.0175	1.158	75.42
(C) + Command-Conditioned	0.953	0.024	0.0193	1.357	74.35	0.966	0.016	0.0148	1.066	76.64
(D) Full Hierarchical (Ours)	**0.962**	**0.014**	**0.0155**	**1.120**	**76.10**	**0.975**	**0.010**	**0.0122**	**0.948**	**76.85**

gression, causing uncertainty in command-type prediction to propagate to downstream coordinate generation. After introducing primitive-level decomposition (Configuration B), command prediction and parameter expansion are reorganized into two sub-stages with a clear sequential dependency. The separate supervision signals allow the model to optimize structural planning and geometric refinement independently. In this setting, IoU in the zero-shot mode increases by 0.7 percentage points and DTW decreases by 12.8%. Although CD rises slightly, the overall results indicate that the structured reorganization of the sequence largely mitigates optimization interference between the two tasks.

The introduction of command-conditioned parameter decoding (Configuration C) yields a further substantial improvement. By explicitly injecting the command embedding $e(c_i)$ into the parameter prediction process, the model dynamically adjusts the distribution of coordinate prediction according to the semantics of the current command. This design reduces MSE by 20.0% relatively in the zero-shot setting, from 0.030 to 0.024, indicating that command-type information directly improves coordinate accuracy.

The full configuration (Configuration D) further bridges the information gap between the command prediction stage and the parameter expansion stage through the hierarchical consistency constraint $\mathcal{L}_{\text{hc}}$. By applying regularization with the primitive-start hidden state as the anchor, $\mathcal{L}_{\text{hc}}$ effectively suppresses the accumulated drift of terminal control points in long parameter sequences, further reducing DTW to 1.120 in the zero-shot setting.

More importantly, the comparison in the few-shot setting directly addresses the compatibility between external topological guidance and internal decoding constraints. The results show that, when a strong external reference signal is introduced, the baseline performance of Configuration A improves markedly, with DTW decreasing from 1.620 to 1.280, which indicates that the external reference indeed absorbs part of the burden of structural guidance. In this case, further introducing internal hierarchical decoding constraints, from Configuration B to D, does not cause optimization conflict, and all metrics continue to improve steadily. Ultimately, the internal constraints and external guidance complement each other well, yielding the best overall performance in the few-shot setting, with DTW reduced to 0.948 and RR increased to 76.85%.

O.4. Generalizability of Hierarchical SVG Decoding Across Multiple MLLMs

To further verify the generality and robustness of the hierarchical SVG decoding strategy, we extend it as a plug-and-play component to six representative baseline architectures in current multimodal large language models (MLLMs). The experiments are conducted under the Few-Shot Reference-Guided setting. By comparing each baseline under two configurations, standard autoregressive generation (Standard) and hierarchical decoding integration (+ Hierarchical), we quantitatively analyze the benefit of this strategy across heterogeneous model architectures.

As shown in Tab. 18, the hierarchical decoding strategy consistently yields significant gains on all evaluated models. In terms of geometric topology, by decoupling path command prediction from coordinate prediction, models that are originally limited in modeling complex strokes, such as StarVector-8B and InternSVG-8B, achieve substantial DTW reductions of 42.3% and 38.6%, respectively. In terms of visual fidelity, due to the hierarchical consistency constraint that suppresses coordinate drift, the strong baseline Qwen3-VL-8B further improves its IoU to 0.958. Notably, after integrating this strategy, all models improve the control-point reduction rate (RR) by 3.7% to 5.2%. This broad performance gain strongly demonstrates that hierarchical decoding effectively mitigates the conflict between structural planning and geometric refinement, making it a general solution for improving vector graphic generation quality.

Table 18. Quantitative comparison of Hierarchical SVG Decoding generalizability across six state-of-the-art MLLM baselines under the **Few-Shot Reference-Guided** setting. "+ Hierarchical" indicates the integration of our proposed hierarchical decoding strategy.

Base Model	Variant	Visual Metrics		Topological Metrics		
		IoU ↑	MSE ↓	CD ↓	DTW ↓	RR (%) ↑
StarVector-8B	Standard	0.824	0.068	0.1487	3.644	62.24
	+ Hierarchical	**0.898**	**0.035**	**0.0755**	**2.102**	**67.35**
InternSVG-8B	Standard	0.845	0.058	0.1196	3.035	64.66
	+ Hierarchical	**0.912**	**0.028**	**0.0582**	**1.865**	**69.12**
DeepSeek-VL-7B	Standard	0.869	0.046	0.0927	2.795	66.45
	+ Hierarchical	**0.925**	**0.022**	**0.0455**	**1.688**	**70.50**
Qwen2.5-VL-3B	Standard	0.893	0.039	0.0793	2.343	67.97
	+ Hierarchical	**0.938**	**0.019**	**0.0382**	**1.455**	**71.88**
InternVL3-8B	Standard	0.904	0.033	0.0675	1.892	68.03
	+ Hierarchical	**0.946**	**0.016**	**0.0295**	**1.240**	**72.65**
Qwen3-VL-8B	Standard	0.938	0.024	0.0339	1.496	69.45
	+ Hierarchical	**0.958**	**0.014**	**0.0198**	**1.082**	**73.20**
VFRefine (Ours)	Standard	0.945	0.025	0.0210	1.280	74.50
	+ Full Hierarchical	**0.975**	**0.010**	**0.0122**	**0.948**	**76.85**

O.5. Necessity of Progressive Training Strategy

The training pipeline of VFRefine is organized into three progressive stages, each targeting a different level of learning. To systematically evaluate the necessity of the stage-wise training strategy and clarify the specific contribution of

each stage to the final performance, we construct the following ablation variants by removing individual stages. For **w/o Stage I**, the language model skips pretraining on native SVG text and directly proceeds to the cross-modal alignment in Stage II and the refinement training in Stage III. This setting examines the model's ability to generate complex structures and interpret contextual references without internal geometric and topological priors. For **w/o Stage II**, the model completes Stage I but bypasses the raster-vector alignment task and directly performs refinement training on paired data in Stage III. This setting evaluates the role of prior alignment between pixel space and vector parameter space in the downstream optimization task.

Table 19. Ablation study on the training protocol. We evaluate the impact of removing Stage I (SSP) and Stage II (R2V) on Few-Shot and Zero-Shot performance, as well as training efficiency. "Convergence" denotes the number of iterations required to reach 90% IoU.

Training Protocol	Few-Shot (Ref-Guided)		Zero-Shot (Universal)			Convergence (Iterations) ↓
	IoU ↑	CD ↓	IoU ↑	MSE ↓	CD ↓	
w/o Stage I	0.852	0.0580	0.938	0.025	0.0250	40k
w/o Stage II	0.946	0.0275	0.892	0.041	0.0682	>112k
Full Strategy	**0.975**	**0.0122**	**0.958**	**0.016**	**0.0149**	**35k**

The results in Table 19 reveal distinct impact patterns for the two stages. For Stage I, its removal has a relatively limited effect in the zero-shot setting but causes severe performance degradation in the few-shot setting. The reason is that the few-shot setting relies heavily on the model's ability to structurally parse reference SVG code in the context and perform analogical reasoning. Without the syntax pretraining in Stage I, the model cannot adequately understand the compositional patterns of parameters in complex Bézier curves or effectively extract topological priors from the reference sequences, which ultimately leads to a substantial drop in the quality of reference-guided generation.

Removing Stage II has clear negative effects on both final performance and training efficiency. From the perspective of training convergence, the model with the full three-stage strategy reaches 90% IoU in only 35k iterations, whereas the w/o R2V variant without Stage II still fails to converge to the same level after 112k iterations. This comparison indicates that, without pixel-vector alignment pretraining, forcing the model to learn vectorized representation and topological optimization jointly from scratch leads to a difficult cold-start problem. The alignment between pixel space and vector parameter space established in Stage II provides stable geometric anchors for the refinement task in Stage III, allowing the model to avoid local optima more effectively when handling complex topological reorganization.

O.6. Effectiveness of Hybrid Attention Mechanism

The Chinese glyph optimization task imposes two competing requirements on the visual encoder. First, control-point localization for Bézier curves and the capture of subtle turning patterns rely on high-resolution fine-grained feature extraction. Second, the topological relations among Chinese character strokes span distant regions, which requires a global receptive field to establish effective long-range structural dependencies. To satisfy both requirements, we design a hybrid attention mechanism in Section G. In shallow layers, progressively enlarged window attention focuses on local geometric information. In deep layers, alternating global attention and local window attention jointly support topological modeling and computational efficiency.

To evaluate the effectiveness and efficiency of this design, we construct three attention configurations for ablation.**Full Global Attention:** The window constraint is removed from all 16 Transformer blocks of the visual encoder, and each layer performs self-attention over the full feature map, which corresponds to the standard ViT scheme. **Local Window-Only:** The global attention layers in deep blocks are removed, and all layers use only fixed-window local attention. As a result, the receptive field is strictly bounded by the window, and there is no direct channel for information exchange across windows. **Hybrid Attention (Ours):** We adopt the proposed hybrid design. The shallow layers use progressive window sizes of $(2, 4, 8)$ to focus on high-frequency geometric details, while the deep layers alternate between 16×16 local window attention and global attention, achieving a balance between topological modeling capacity and computational cost.

Tab. 20 provides a quantitative comparison of the three configurations in terms of generation performance and resource consumption. Full Global Attention achieves slightly better IoU and CD scores, but at a substantially higher computational cost, which imposes considerable resource pressure in practical deployment. Local Window-Only is clearly more efficient, but its limited receptive field prevents the model from effectively capturing long-range topological relations among strokes, leading to consistent degradation across generation metrics. By contrast, Hybrid Attention achieves a favorable balance between performance and efficiency. Small windows in shallow layers ensure precise extraction of local geometric features, while the sparsely placed global attention layers in deep blocks provide the necessary pathways for modeling long-range cross-region dependencies. Under this design, the model incurs only a small additional computational cost while achieving generation quality that is nearly identical to that of Full Global Attention.

Table 20. Ablation study on the Attention Mechanism under the Few-Shot Reference-Guided setting. We compare the proposed Hybrid Attention against Full Global and Local Window-Only baselines.

Mechanism	Performance		Efficiency	
	IoU ↑	CD ↓	Peak Mem (GB) ↓	Latency (ms) ↓
Full Global Attention	0.970	0.0119	20.8	3,746
Local Window-Only	0.937	0.0319	**5.9**	2,139
Hybrid Attention (Ours)	**0.972**	**0.0122**	**9.6**	**1,812**

O.7. Generalizability of Task-Aware Head Specialization

To examine the applicability of the proposed task-aware attention head specialization mechanism across different model architectures and its portability as a plug-and-play component, we extend it to all mainstream multimodal large language model (MLLM) baselines considered in this study. In the autoregressive SVG generation framework, the classification of discrete path commands and the regression optimization of continuous coordinate values involve

conflicting objectives, which limits overall generation quality. Based on this observation, we integrate the head-level routing strategy into the last three decoder layers of each baseline model and evaluate all modified baselines together with the proposed VFRefine under the Zero-Shot Universal setting using a unified hyperparameter configuration for a fair comparison.

Table 21. Quantitative comparison demonstrating the generalizability of the Task-Aware Head Specialization mechanism across all MLLM baselines, including our VFRefine. "+ Head Spec." denotes the integration of the proposed routing mechanism.

Method	Variant	Visual Metrics		Topological Metrics		
		IoU ↑	MSE ↓	CD ↓	DTW ↓	RR (%) ↑
StarVector-8B	Standard	0.808	0.077	0.1480	3.657	62.19
	+ **Head Spec.**	**0.824**	**0.068**	**0.1363**	**3.330**	**65.44**
InternSVG-8B	Standard	0.835	0.068	0.1262	3.197	65.76
	+ **Head Spec.**	**0.841**	**0.055**	**0.1058**	**2.900**	**67.92**
DeepSeek-VL-7B	Standard	0.857	0.049	0.0938	2.589	68.07
	+ **Head Spec.**	**0.866**	**0.041**	**0.0815**	**2.366**	**69.50**
Qwen2.5-VL-3B	Standard	0.870	0.044	0.0733	2.205	69.48
	+ **Head Spec.**	**0.885**	**0.037**	**0.0680**	**2.103**	**70.08**
InternVL3-8B	Standard	0.899	0.038	0.0610	1.995	70.82
	+ **Head Spec.**	**0.910**	**0.030**	**0.0496**	**1.881**	**71.43**
Qwen3-VL-8B	Standard	0.926	0.025	0.0339	1.569	71.85
	+ **Head Spec.**	**0.942**	**0.023**	**0.0280**	**1.453**	**72.92**
VFRefine (Ours)	Standard	0.954	0.026	0.0188	1.230	74.33
	+ **Head Spec.**	**0.965**	**0.020**	**0.0169**	**1.255**	**75.33**

The quantitative results in Tab. 21 show that the task-aware attention head specialization mechanism consistently improves performance across all evaluated architectures. In terms of geometric accuracy, both the mean squared error (MSE) and the Chamfer distance (CD) decrease for all model variants, indicating a general improvement in coordinate prediction accuracy and an effective reduction of the coordinate drift commonly observed in standard autoregressive SVG generation. In terms of sequence quality, all baselines also improve on compression ratio (RR) and dynamic time warping (DTW), which suggests that specialized attention heads better distinguish the feature requirements of command-type prediction and coordinate regression, thereby guiding the model to generate command sequences that are more compact and better aligned in topology.

Among all evaluated models, VFRefine with this mechanism achieves the best overall performance, with IoU increasing to 0.965 and MSE decreasing to 0.020. These cross-architecture results indicate that the task-aware head-level routing mechanism is not a customized technique tailored to a specific model structure, but a general optimization strategy decoupled from architecture design. Its role is to mitigate the joint optimization conflict between discrete prediction and continuous regression in vector graphics generation at the level of attention allocation.

O.8. Ablation on Loss Function Components

The final training objective consists of the standard autoregressive loss $\mathcal{L}_{\text{svg}}$ together with four auxiliary loss terms. To verify the necessity of each component, we design a set of progressive ablation experiments under the Zero-Shot Universal setting.

The results in Tab. 22 reflect the performance changes as the loss terms are introduced progressively. When training relies only on the autoregressive loss (Configuration A), the model lacks explicit supervision to distinguish the two heterogeneous tasks, namely discrete topology compression and continuous coordinate regression, and the optimization conflict between them is not effectively alleviated. Directly adding task-specific auxiliary losses on this basis (Configuration B) yields only limited gains, because the attention heads have not yet developed clear functional specialization. As a result, gradient signals from different tasks interfere within the same set of attention heads, and the auxiliary losses cannot accurately transmit their optimization directions to the corresponding computational units.

Table 22. Ablation study on the components of the total loss function. Progressive addition of task-specific losses ($\mathcal{L}_{\text{topo}}$, $\mathcal{L}_{\text{coord}}$), separation regularization ($\mathcal{L}_{\text{sep}}$), and load balancing ($\mathcal{L}_{\text{bal}}$) consistently improves performance.

Loss Configuration	IoU ↑	MSE ↓	CD ↓	DTW ↓	RR (%) ↑
(A) $\mathcal{L}_{\text{svg}}$ only (Baseline)	0.948	0.025	0.0191	1.262	74.34
(B) (A) + $\tilde{\mathcal{L}}_{\text{topo}} + \tilde{\mathcal{L}}_{\text{coord}}$	0.957	0.024	0.0173	1.207	75.86
(C) (B) + $\mathcal{L}_{\text{sep}}$	0.968	0.022	0.0158	1.144	76.14
(D) Full	**0.971**	**0.020**	**0.0169**	**1.255**	**75.33**

Introducing the entropy regularization term $\mathcal{L}_{\text{sep}}$ (Configuration C) encourages the attention heads to specialize, establishing a clearer division of roles between discrete command prediction and continuous coordinate regression, which leads to significant improvements in both topological compactness (RR) and geometric accuracy (CD). However, without the load-balancing constraint, we observe an imbalance: the coordinate regression task produces denser gradient signals, causing too many attention heads to collapse to that branch and leaving insufficient specialized computational resources for discrete topology compression. The full configuration (Configuration D) resolves this issue by introducing the balancing term $\mathcal{L}_{\text{bal}}$, which constrains the number of attention heads assigned to each task branch. This term effectively prevents task collapse, maintains a reasonable distribution of specialized heads across tasks, and enables the model to achieve the best generation quality.

P. ANALYSIS

P.1. Sensitivity to Number of Reference Templates

In the Few-Shot Reference-Guided mode, the number of reference samples K determines how much topological prior information is available to the model. To identify the optimal reference set size that balances generation quality and computational efficiency, we evaluate the full range $K \in \{0, 1, \ldots, 8\}$ on the VFRefine-1.2M test set, where $K = 0$ degenerates to the Zero-Shot Universal mode without reference input.

The results in Fig. 13 show a trend of initial improvement followed by saturation. As K increases from 0 to 3, the model performance improves steadily and reaches its peak at $K = 3$. When K exceeds 3, adding more reference samples does not yield further quality gains and instead causes slight performance fluctuations. A plausible explanation is that excessive reference templates inject redundant visual information and noisy features into the model, which weakens the ability of the attention mechanism to focus on geomet-

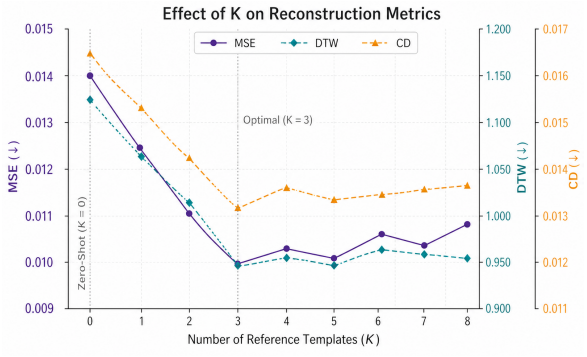


Fig. 13. Impact of reference template count (K) on geometric error metrics. $K = 0$ corresponds to the Zero-Shot Universal results in Tab. 9. We observe that error metrics decrease rapidly and reach a global minimum at $K = 3$, verifying it as the optimal setting for reference-guided refinement.

ric structure while also increasing unnecessary inference cost. Based on these results, we set the default number of reference samples in the Reference-Guided mode to $K = 3$, which effectively exploits the topological prior information in the references at low computational cost.

P.2. High-Fidelity Raster-to-Vector Conversion

Although VFRefine is primarily designed for vector glyph optimization, the cross-modal alignment established in its second training stage (Stage II: Raster-to-Vector Alignment) provides the basis for mapping from raster pixel space to vector parameter space. This stage is optimized by maximizing the conditional probability $p_\theta(S_{\text{des}} \mid \mathcal{E}_{\text{vis}}(I_{\text{src}}))$, under which the visual encoder learns to extract glyph geometric features and translate them into standardized SVG code representations. This alignment from raster perception to vector generation supports the visual understanding required by the subsequent topology optimization stage and also endows the model with the potential for end-to-end image vectorization.

To verify this capability, we design the following evaluation protocol. We directly feed standard Chinese character raster images into the model without providing any reference SVG code as an auxiliary condition, and require the model to independently reconstruct the complete SVG XML sequence in an end-to-end manner from the raster input. Fig. 14 shows the overlay comparison between the original input and the model output after re-rendering the generated SVG code into image space. The visualization indicates that the Bézier curves generated by VFRefine align well with the original raster stroke boundaries. At the same time, the model effectively reduces the redundant control nodes commonly produced by traditional vectorization methods while preserving geometric accuracy, yielding more compact and smoother vector contour structures.

P.3. Glyph Image Vectorization

The VFRefine framework shows promising generalization potential in vector processing and covers two types of vector tasks. The first is Vector Glyph Refinement, which is the core task of this paper. It evaluates whether the model can reconstruct geometrically redundant and non-canonical inputs into compact SVG representations while preserving visual consistency with the original input. The second is Image-to-Vector Generation. This task leverages the cross-modal alignment prior learned during training to assess whether the model can perform end-to-end vector reconstruction from raster images alone.

Tab. 23 reports a quantitative comparison on glyph image vectorization between optimization-based methods and learning-based methods. Optimization-based methods, such as Im2Vec and VecFontSDF, as well as early sequence generation models, such as DeepSVG and the DeepVecFont series, exhibit relatively large errors on both geometric and topological metrics (CD and DTW). The main reason is that, when the input is only a single image, these methods lack a mechanism to impose direct structural constraints on vector paths. VecFusion achieves stronger performance by combining contour priors with pixel texture information. Trained on the VFRefine-1.2M dataset, VFRefine further outperforms all these methods on the overall metrics. Specifically, VFRefine achieves the highest IoU of 0.926 and the lowest values on MSE (0.0382), Chamfer Distance (0.0745), and Dynamic Time Warping (2.812). These results show that VFRefine can directly reconstruct geometrically accurate and topologically organized vector paths from raster images.

Table 23. Quantitative comparison of image-to-vector glyph reconstruction methods. The table evaluates optimization-based and learning-based approaches across visual (IoU, MSE) and topological (CD, DTW) metrics.

Methods	Vec. Superv.	Img-to-vec	IoU ↑	MSE ↓	CD ↓	DTW ↓
Im2Vec [24]	×	✓	0.769	0.1028	0.1910	5.589
VecFontSDF [43]	×	✓	0.774	0.0930	0.1639	5.046
DeepSVG [5]	✓	×	0.730	0.1052	0.2016	5.319
DeepVecFont [37]	✓	✓	0.829	0.0663	0.1498	4.630
DeepVecFont-v2 [38]	✓	✓	0.870	0.0529	0.1169	3.701
VecFusion [30]	✓	✓	0.912	0.0423	0.0917	3.385
VFRefine (Ours)	✓	✓	**0.926**	**0.0382**	**0.0745**	**2.812**

P.4. Vector Glyph Refinement via Shuffled Component Reference

To examine whether VFRefine can maintain effective refinement when the reference sample and the target glyph lack global structural correspondence, we construct a robustness experiment that uses disordered components as reference inputs. As shown in Fig. 15, we decompose the reference character into stroke components and randomly shuffle their spatial positions, producing deformed reference samples in which the global semantic structure is destroyed while the local geometric features remain consistent with the original reference.

Under this setting, the spatial correspondence between the reference image and the input glyph no longer exists, so the model cannot obtain refinement cues through strategies that rely on spatial consistency, such as feature matching or optical flow alignment. The results show that even when the spatial arrangement of the reference character is completely disordered, the model still extracts topological priors of Bézier curves from the discrete components and accurately applies them to the vector reconstruction of the target glyph. This suggests that VFRefine does not rely on global struc-

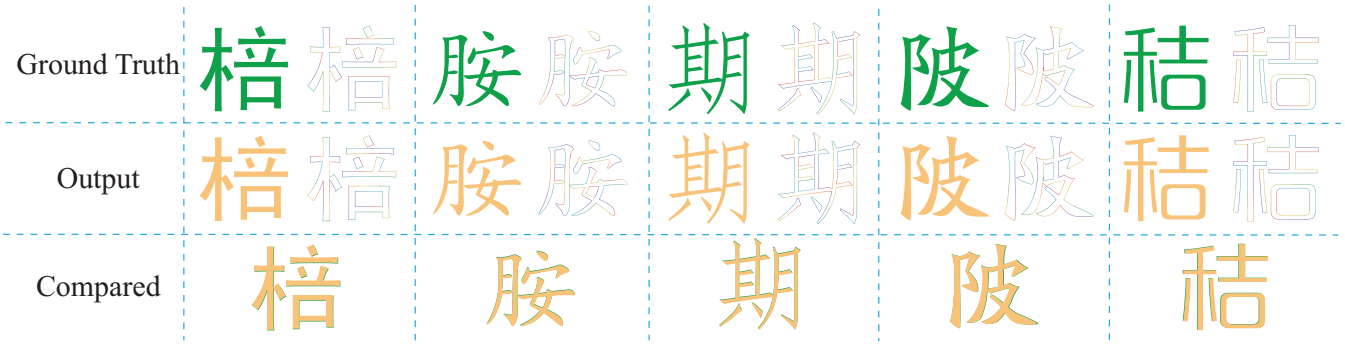


Fig. 14. Visualization of image-to-vector generation results. Given a raster image as input, the model directly generates compact SVG code that preserves sparse vector topology while achieving precise alignment with the original glyph contours.

tural alignment to exploit the topological priors of reference samples, but instead depends on the independent perception and transfer of local geometric features.

P.5. Vector Refinement on Multi-Style Fonts

The preceding experiments validate the performance of VFRefine. This section further presents its compression results across diverse font styles and evaluates its robustness to inputs with substantial morphological variation. As shown in Fig. 16, the test set includes calligraphic and artistic fonts with relatively complex geometric structures. Compared with regular printed fonts, these styles exhibit pronounced domain gaps in stroke width variation, connectivity patterns, and contour curvature distributions, which place higher demands on the model's ability to infer path topology accurately. The results show that VFRefine consistently reorganizes redundant Bézier curve segments in the input into compact geometric representations across a wide style range, from regular printed fonts to free-form calligraphic fonts, while substantially reducing the number of control points. At the same time, the compressed vector contours remain highly consistent with the original raster images in visual appearance, without introducing perceptible shape distortion despite the large reduction in control points.

P.6. Local Topological Perception under Disordered Stroke References

Whether the refinement process of VFRefine is grounded in rigid matching to the global semantics of the reference image or in fine-grained learning of local geometric and topological rules is central to understanding the model's internal mechanism. To distinguish between these two hypotheses, we design a stroke-level feature deconstruction experiment. Under the standard few-shot reference setting, the model receives structurally complete Chinese character images as reference inputs. This experiment changes that premise by decomposing the reference glyph into stroke-level components and randomly rearranging their spatial positions, so that the global structure and semantic information of the reference sample are no longer preserved.

As shown in Fig. 17, even when the reference template degenerates into spatially disordered stroke fragments, VFRefine still captures critical geometric features from them and guides the character under refinement to produce a plausible vector output. This result indicates that the model's refinement mechanism is not based on global template alignment or semantic memorization, but on its ability to decouple spatial information from semantic information. Specifically, the model extracts Bézier curve construction rules from discrete and disordered visual cues and robustly transfers these local topological priors, which are independent of absolute spatial position, to the target glyph, thereby completing the reorganization and reconstruction of vector paths.

P.7. Vector Optimization of Generated Glyphs

Few-shot font generation has made substantial progress in style transfer. However, its outputs are mainly raster images, whose boundaries often exhibit varying degrees of smoothness defects. Converting such images into vector format is necessary for practical use, yet engineering tools for this task, such as Potrace or Adobe Image Trace, cannot identify or correct these defects. The reason is that these algorithms are built on strict geometric fitting and trace pixelated contours point by point, without the ability to distinguish valid structures from noise. As a result, they encode artifacts in generated images as vector path control points. The resulting vector glyphs therefore fail to meet professional design standards in both boundary smoothness and control point compactness.

To address this issue, we use font images generated by FC-Font[15] as test inputs and evaluate the ability of VFRefine to correct non-smooth contours. Unlike the point-wise tracing strategy of conventional tools, VFRefine formulates vectorization as a reconstruction task driven by semantic understanding. It relies on implicit design priors learned from high-quality vector datasets to distinguish intended stylistic features from unintended generation artifacts. As shown in Fig. 18, the results show that the reconstruction is not constrained by the rough pixel boundaries of the input image, but instead restores glyph contours with smooth and continuous Bézier curves. After refinement by VFRefine, the number of control points in the vector glyph is substantially reduced, and the inherent contour irregularities of generative images are corrected. This result verifies the practical value of VFRefine as a post-processing module, enabling it to bridge rasterized creative generation and vectorized indus-

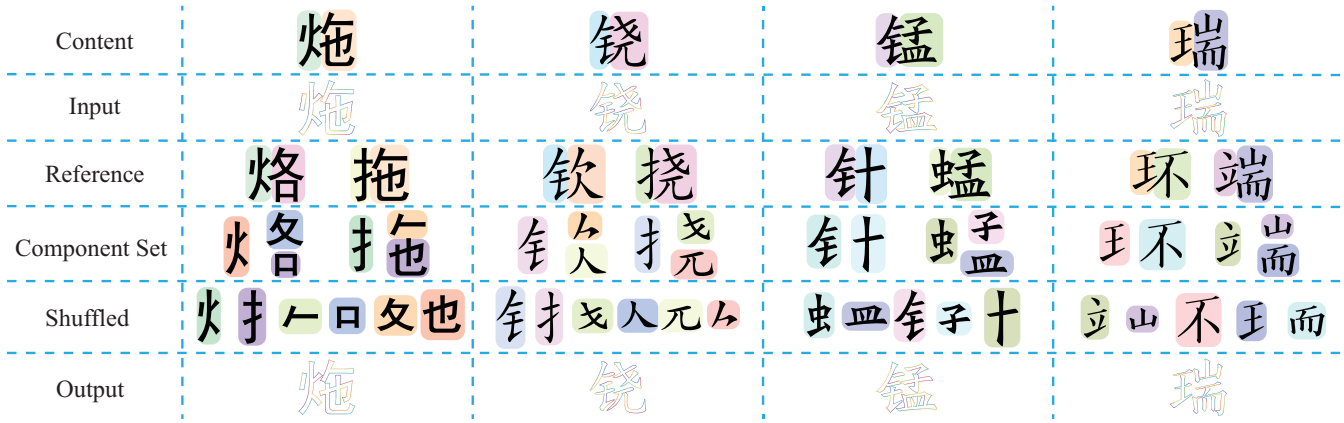


Fig. 15. Qualitative experiment on vector glyph optimization based on disordered component references.

trial production.

P.8. Elimination of Generative Artifacts

In font generation, images produced by both GAN-based and diffusion-based methods contain isolated pixel noise and visual artifacts to varying degrees. For such noisy inputs, traditional vectorization tools represented by Potrace cannot perform semantic discrimination because their underlying algorithms strictly follow the geometric logic of contour tracing. Consequently, they parameterize isolated pixel clusters that carry no semantic information as independent closed SVG paths. These redundant geometric primitives not only compromise the structural cleanliness of vector glyphs but also introduce additional costs for subsequent editing and rendering, which limits the practical usability of generative font libraries in industrial settings.

By contrast, VFRefine exhibits denoising behavior during vector reconstruction through its structural priors and semantic perception mechanism. Rather than fitting boundaries point by point as in conventional algorithms, VFRefine formulates vectorization as a structure reorganization task based on semantic understanding. When processing noisy samples generated by SOTA font generation models[15], the model uses the Chinese character topology learned from training data to distinguish valid stroke structures from randomly generated noise. As shown in Fig. 19, even when the input raster image contains pronounced isolated artifacts, VFRefine automatically filters out these unstructured visual disturbances during SVG sequence prediction and reconstructs only the valid paths that form the main body of the character.

P.9. Implicit Consistency of Radicals

In font engineering practice, the geometric consistency of radicals across different characters is a core criterion for evaluating font quality. When the same component appears in different Chinese character structures, its shape, proportion, and stroke details should maintain stable visual uniformity. Therefore, an ideal vector refinement model should standardize the same component across different contexts.

To evaluate VFRefine on this aspect, we select several non-canonical characters that contain the same radical but exhibit different degrees of geometric distortion and noise as test inputs. As shown in Fig. 20, the outputs processed by VFRefine exhibit strong geometric consistency across characters. The corresponding radicals in different characters converge to a unified visual style and retain high similarity in the layout logic of Bézier control points. This result indicates that VFRefine does not optimize each character as an isolated unit. Instead, during training, it encodes the modular design regularities of Chinese character components as implicit knowledge, which enables cross-character standardization of component style at inference time.

P.10. Restoration of Structural Defects

In practical font design and vectorization pipelines, the quality issues of vector glyphs are not limited to geometric redundancy. Manual editing with interactive tools also introduces topological defects. A typical case is the misoperation of Bézier curve control handles. When the handle position or direction deviates, the corresponding stroke width undergoes unintended abnormal fluctuations, which further causes local geometric collapse and visual distortion of the glyph. Unlike non-canonical geometric issues such as redundant control points, such defects constitute damage to the topological structure of the vector path.

To evaluate the ability of VFRefine to repair such structural errors, we collect vector glyph samples with these topological defects from real designer workflows as test inputs. These samples differ from non-canonical glyphs that contain only redundant nodes: the latter still preserve intact geometric structure, whereas the former already exhibit broken path structure. As shown in Fig. 21, the experiment directly feeds vector glyphs with the above defects into the model and requires it to output canonical SVG code. The results show that VFRefine not only optimizes redundant structures but also repairs damaged geometry. Rather than mechanically fitting the erroneous geometric features in the input, the model identifies and corrects defective regions according to the structural priors of Chinese characters learned from high-quality vector data during training.

Fig. 16. The results on complex calligraphic and artistic fonts show that VFRefine preserves the visual aesthetics and stylistic characteristics of highly deformed glyphs while reconstructing vector glyphs with compact geometric representations and optimized topological structures.

P.11. Cross-Script Vector Glyph Optimization

To examine the generalization of the proposed framework across writing systems, we extend the evaluation from Chinese characters to English, Korean, and Japanese characters that are not seen during training. In the experiments, we use Potrace to convert raster images from these writing systems into initial vector representations, and provide the resulting SVG files to the model as non-canonical inputs. As shown in Fig. 22, the vectorized results produced by Potrace contain substantial geometric redundancy, with densely distributed control points and little structured organization. On this basis, we directly apply VFRefine, which is trained only on Chinese character data, to these cross-lingual inputs without any fine-tuning or adaptation during inference. The results show that the geometric optimization priors learned during training transfer effectively to unseen writing systems: redundant segments are removed, path topologies are compactly reorganized, and glyph visual fidelity is preserved. This suggests that the vector optimization knowledge learned by VFRefine is not limited to structure patterns specific to Chinese characters, but captures geometric regularities of canonicalization shared across writing systems.

P.12. Vector Icon Optimization

We further extend the evaluation of VFRefine to the domain of general planar vector icons to examine the boundary of its cross-domain generalization. Compared with the Chinese glyph data used in training, planar icons differ substantially in geometric composition, semantic structure, and path organization, which makes this setting a more challenging transfer scenario. In the experiments, we again use Potrace to convert raster icon images into initial vector representations. As shown in Fig. 23, the resulting vector paths contain clear redundancy. Under this setting, we directly ap-

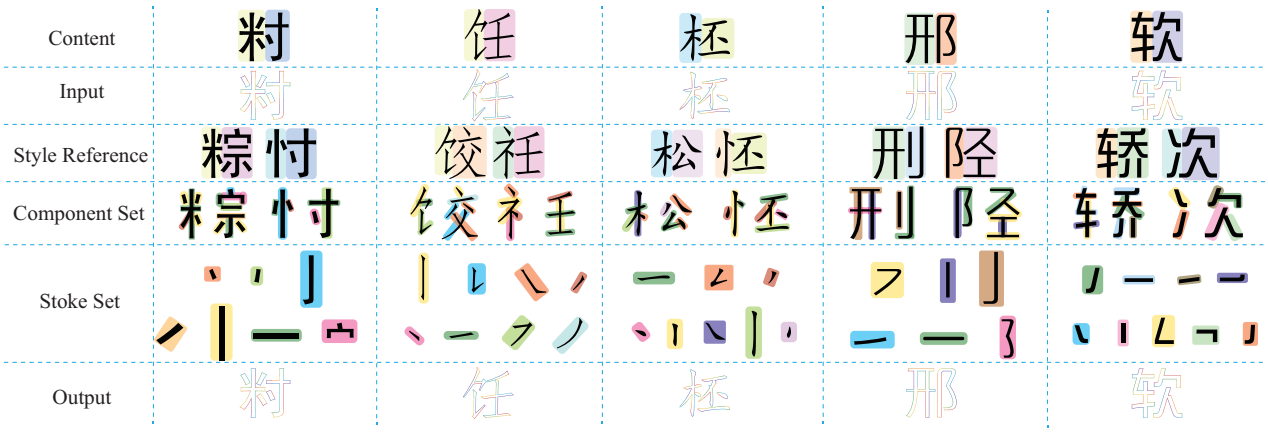


Fig. 17. Visualization of stroke-level topology perception under spatial disorder. In this example, the original reference character is decomposed into mutually independent stroke paths, which are then randomly rearranged to destroy the global structural relationships in the reference character. Even when the reference template degenerates into an unordered set of stroke fragments, VFRefine still extracts effective geometric cues from these discrete components and uses them to guide the reconstruction of the irregular glyph.

Fig. 18. Glyph vector refinement for few-shot font generation results. The input samples are raster glyphs synthesized by a few-shot font generation model, whose contours exhibit non-smooth artifacts. While preserving the original glyph structure and stylistic consistency, VFRefine reconstructs high-quality vector glyphs with smooth outer contours and compact geometric representations.

ply VFRefine, without any fine-tuning on icon data, to these non-canonical inputs. The results show that the model effectively infers the geometric logic of the inputs and reorganizes redundant vector paths into concise parameterized geometric shapes. This indicates that the topological optimization priors learned by VFRefine during training are not limited to the specific domain of Chinese glyphs, but also transfer to broader categories of vector graphics.

P.13. Failure Case Analysis

VFRefine shows strong performance on vector optimization for standard printed fonts, but it also exhibits clear limitations when the input shifts from canonical printed glyphs to highly stylized calligraphic fonts. As shown in Fig. 24, for calligraphic glyphs with extremely rough edges, the vector outputs lose detail in contour reconstruction, particularly the details that carry the artistic expressiveness of the original glyphs.

This limitation stems from a substantial mismatch between the training data distribution and the geometric properties of calligraphic fonts. The VFRefine-1.2M dataset is built from standardized fonts that satisfy industrial production requirements, whose defining characteristics are smooth contour curves and clear geometric boundaries. Training on this dataset induces a strong smoothing bias in the model. Specifically, the model tends to treat high-frequency contour fluctuations as geometric redundancy or noise that should be removed. However, the rough edges in calligraphic fonts are not meaningless geometric noise. Instead, they are semantic features that convey brush texture and writing rhythm. When processing such inputs, the topological optimization strategy learned during training acts on these artistically meaningful high-frequency textures and fits them with low-

Fig. 19. Removal of generation artifacts. The input raster image contains floating noise introduced by the generative model, which carries no glyph semantic information. During vector reconstruction, VFRefine effectively suppresses these non-semantic artifacts and thus produces clean and practically usable vector glyphs.

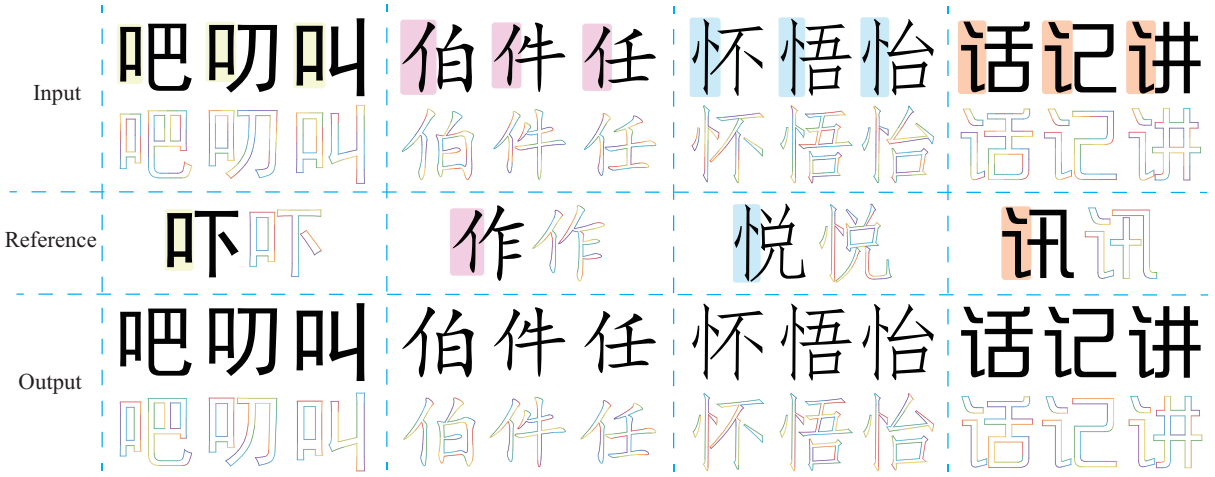


Fig. 20. Visual verification of component consistency. For identical components that recur across different characters, VFRefine produces results with consistent geometric forms and more standardized structural expressions.

frequency smooth Bézier curves, which systematically erases texture information during optimization.

This oversmoothing indeed produces highly compact path structures in geometric terms, but at the cost of the stylistic characteristics and artistic expressiveness inherent to calligraphic glyphs. This observation indicates an inherent trade-off between pursuing extreme vector compression and preserving complex artistic details. Future work may address this issue by expanding the dataset with calligraphic vector data.

Fig. 21. Qualitative results of structural defect repair. For topological anomalies in input glyphs, VFRefine identifies and corrects them using learned structural priors, and restores the damaged glyphs to standard representations that conform to canonical forms.

Fig. 22. Qualitative evaluation of cross-lingual zero-shot generalization. Although VFRefine is trained only on a Chinese character dataset, it still shows strong adaptability to unseen writing systems, including Latin letters, Korean, and Japanese characters.

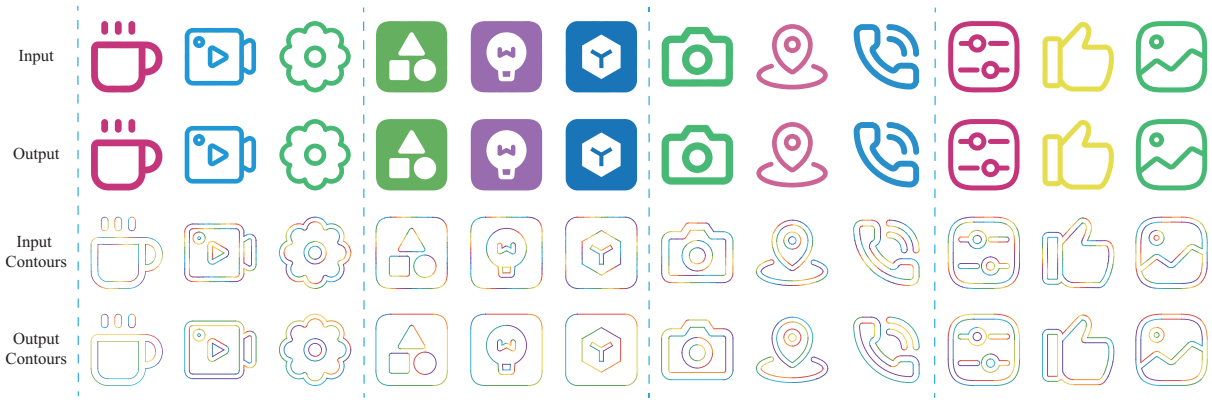


Fig. 23. Zero-shot generalization results on vector icons. Although VFRefine is trained only on Chinese glyph data, the implicit geometric priors it learns transfer to the processing of general planar vector graphics.

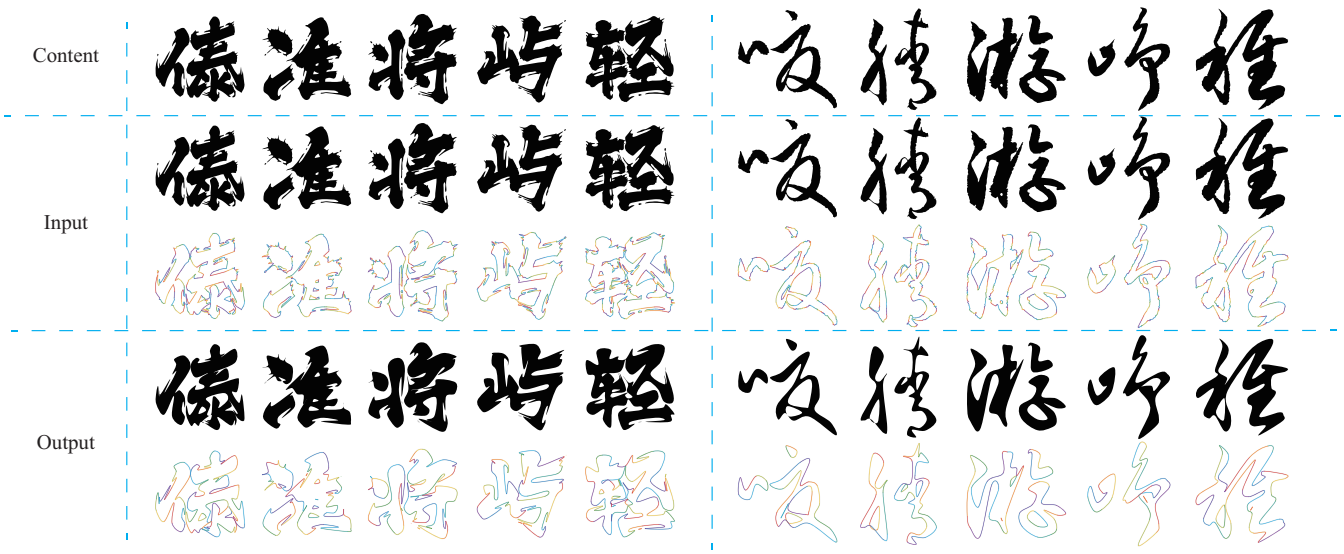


Fig. 24. Failure cases under artistic calligraphic styles. Due to the lack of calligraphic samples in the training data, VFRefine struggles to distinguish artistically rough edges from geometric noise that should be corrected when processing such highly stylized glyphs. As a result, the model tends to apply overly smooth corrections, which weakens the original writing style and yields oversimplified vector contours.

Q. REFERENCES

[1] Bai, S., Cai, Y., Chen, R., Chen, K., Chen, X., Cheng, Z., Deng, L., Ding, W., Gao, C., Ge, C., et al.: Qwen3-vl technical report. arXiv preprint arXiv:2511.21631 (2025)

[2] Bai, S., Chen, K., Liu, X., Wang, J., Ge, W., Song, S., Dang, K., Wang, P., Wang, S., Tang, J., et al.: Qwen2. 5-vl technical report. arXiv preprint arXiv:2502.13923 (2025)

[3] Bisong, E., et al.: Building machine learning and deep learning models on Google cloud platform. Springer (2019)

[4] Cai, M., Huang, Z., Li, Y., Ojha, U., Wang, H., Lee, Y.J.: Leveraging large language models for scalable vector graphics-driven image understanding. arXiv preprint arXiv:2306.06094 (2023)

[5] Carlier, A., Danelljan, M., Alahi, A., Timofte, R.: Deepsvg: A hierarchical generative network for vector graphics animation. Advances in Neural Information Processing Systems **33**, 16351–16361 (2020)

[6] Fan, H., Su, H., Guibas, L.J.: A point set generation network for 3d object reconstruction from a single image. In: Proceedings of the IEEE conference on computer vision and pattern recognition. pp. 605–613 (2017)

[7] Frans, K., Soros, L., Witkowski, O.: Clipdraw: Exploring text-to-drawing synthesis through language-image encoders. Advances in Neural Information Processing Systems **35**, 5207–5218 (2022)

[8] Gat, I., Remez, T., Shaul, N., Kreuk, F., Chen, R.T., Synnaeve, G., Adi, Y., Lipman, Y.: Discrete flow matching. Advances in Neural Information Processing Systems **37**, 133345–133385 (2024)

[9] Husain, H., Wu, H.H., Gazit, T., Allamanis, M., Brockschmidt, M.: Codesearchnet challenge: Evaluating the state of semantic code search. arXiv preprint arXiv:1909.09436 (2019)

[10] Jain, A., Xie, A., Abbeel, P.: Vectorfusion: Text-to-svg by abstracting pixel-based diffusion models. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. pp. 1911–1920 (2023)

[11] Lee, K., Joshi, M., Turc, I.R., Hu, H., Liu, F., Eisenschlos, J.M., Khandelwal, U., Shaw, P., Chang, M.W., Toutanova, K.: Pix2struct: Screenshot parsing as pretraining for visual language understanding. In: International Conference on Machine Learning. pp. 18893–18912. PMLR (2023)

[12] Li, J., Li, D., Savarese, S., Hoi, S.: Blip-2: Bootstrapping language-image pre-training with frozen image encoders and large language models. In: International conference on machine learning. pp. 19730–19742. PMLR (2023)

[13] Li, T.M., Lukáč, M., Gharbi, M., Ragan-Kelley, J.: Differentiable vector graphics rasterization for editing and learning. ACM Transactions on Graphics (TOG) **39**(6), 1–15 (2020)

[14] Liu, Y.T., Zhang, Z., Guo, Y.C., Fisher, M., Wang, Z., Zhang, S.H.: Dualvector: Unsupervised vector font synthesis with dual-part representation. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. pp. 14193–14202 (2023)

[15] Liu, Y., Ding, Y., Khalid, F.B., Wang, C., Wang, L.: Few-shot font generation via denoising diffusion and component-level fine-grained style. Expert Systems with Applications **296**, 128987 (2026)

[16] Liu, Y., Fatimah, B.K., Cunrui, W., Mas Rina, B.M., Azreen, B.A.: Diffvecfont: Fusing dual-mode reconstruction vector fonts via masked diffusion transformers. In: Computational Visual Media. pp. 339–363. Springer Nature Singapore, Singapore (2025)

[17] Liu, Y., binti Khalid, F., binti Mustaffa, M.R., bin Azman, A.: Dual-modality learning and transformer-based approach for high-quality vector font generation. Expert Systems with Applications **240**, 122405 (2024)

[18] Lopes, R.G., Ha, D., Eck, D., Shlens, J.: A learned representation for scalable vector graphics. In: Proceedings of the IEEE/CVF International Conference on Computer Vision. pp. 7930–7939 (2019)

[19] Lu, H., Liu, W., Zhang, B., Wang, B., Dong, K., Liu, B., Sun, J., Ren, T., Li, Z., Yang, H., et al.: Deepseek-vl: towards real-world vision-language understanding. arXiv preprint arXiv:2403.05525 (2024)

[20] Nijkamp, E., Pang, B., Hayashi, H., Tu, L., Wang, H., Zhou, Y., Savarese, S., Xiong, C.: Codegen: An open large language model for code with multi-turn program synthesis. arXiv preprint arXiv:2203.13474 (2022)

[21] Radford, A., Kim, J.W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., et al.: Learning transferable visual models from natural language supervision. In: International conference on machine learning. pp. 8748–8763. PmLR (2021)

[22] Radford, A., Narasimhan, K., Salimans, T., Sutskever, I., et al.: Improving language understanding by generative pre-training (2018)

[23] Ramesh, A., Dhariwal, P., Nichol, A., Chu, C., Chen, M.: Hierarchical text-conditional image generation with clip latents. arXiv preprint arXiv:2204.06125 **1**(2), 3 (2022)

[24] Reddy, P., Gharbi, M., Lukac, M., Mitra, N.J.: Im2vec: Synthesizing vector graphics without vector supervision. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. pp. 7342–7351 (2021)

[25] Rodriguez, J.A., Puri, A., Agarwal, S., Laradji, I.H., Rodriguez, P., Rajeswar, S., Vazquez, D., Pal, C., Pedersoli, M.: Starvector: Generating scalable vector graphics code from images and text. In: Proceedings of the Computer Vision and Pattern Recognition Conference. pp. 16175–16186 (2025)

[26] Sakoe, H., Chiba, S.: Dynamic programming algorithm optimization for spoken word recognition. IEEE transactions on acoustics, speech, and signal processing **26**(1), 43–49 (2003)

[27] Selinger, P.: Potrace: a polygon-based tracing algorithm (2003)

[28] Selinger, P.: Potrace: Transform bitmaps into vector graphics. `http://potrace.sourceforge.net` (2003), accessed: 2025-01-01

[29] Song, J., You, W., Shi, S., Guo, S., Sun, L., Wang, W.: Efficient and scalable chinese vector font generation via component composition. arXiv preprint arXiv:2404.06779 (2024)

[30] Thamizharasan, V., Liu, D., Agarwal, S., Fisher, M., Gharbi, M., Wang, O., Jacobson, A., Kalogerakis, E.: Vecfusion: Vector font generation with diffusion. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. pp. 7943–7952 (2024)

[31] Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., et al.: Llama: Open and efficient foundation language models. arXiv preprint arXiv:2302.13971 (2023)

[32] Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., Batra, S., Bhargava, P., Bhosale, S., et al.: Llama 2: Open foundation and fine-tuned chat models. arXiv preprint arXiv:2307.09288 (2023)

[33] Tsang, C.: Vtracer: Open source raster to vector graphics converter. `https://github.com/visioncortex/vtracer` (2020), accessed: 2026-01-22

[34] van Rossum, J., The FontTools Authors: Fonttools: A library to manipulate font files from python. `https://github.com/fonttools/fonttools` (1999), accessed: 2026-01-22

[35] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., Polosukhin, I.: Attention is all you need. Advances in neural information processing systems **30** (2017)

[36] Wang, H., Yin, J., Wei, Q., Zeng, W., Gu, L., Ye, S., Gao, Z., Wang, Y., Zhang, Y., Li, Y., et al.: Internsvg: Towards unified svg tasks with multimodal large language models. arXiv preprint arXiv:2510.11341 (2025)

[37] Wang, Y., Lian, Z.: Deepvecfont: synthesizing high-quality vector fonts via dual-modality learning. ACM Transactions on Graphics (TOG) **40**(6), 1–15 (2021)

[38] Wang, Y., Wang, Y., Yu, L., Zhu, Y., Lian, Z.: Deepvecfont-v2: Exploiting transformers to synthesize vector fonts with higher quality. In: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition. pp. 18320–18328 (2023)

[39] Wang, Z., Bovik, A.C., Sheikh, H.R., Simoncelli, E.P.: Image quality assessment: from error visibility to structural similarity. IEEE transactions on image processing **13**(4), 600–612 (2004)

[40] Weber, M.: Autotrace: A utility for converting bitmap to vector graphics. `http://autotrace.sourceforge.net/` (1998), accessed: 2025-01-22

[41] Wu, R., Su, W., Liao, J.: Chat2svg: Vector graphics generation with large language models and image diffusion models. In: Proceedings of the Computer Vision and Pattern Recognition Conference. pp. 23690–23700 (2025)

[42] Wu, R., Su, W., Ma, K., Liao, J.: Iconshop: Text-guided vector icon synthesis with autoregressive transformers. ACM Transactions on Graphics (TOG) **42**(6), 1–14 (2023)

[43] Xia, Z., Xiong, B., Lian, Z.: Vecfontsdf: Learning to reconstruct and synthesize high-quality vector fonts via signed distance functions. In: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition. pp. 1848–1857 (2023)

[44] Xing, X., Hu, J., Liang, G., Zhang, J., Xu, D., Yu, Q.: Empowering llms to understand and generate complex vector graphics. In: Proceedings of the Computer Vision and Pattern Recognition Conference. pp. 19487–19497 (2025)

[45] Xu, Y., Li, M., Cui, L., Huang, S., Wei, F., Zhou, M.: Layoutlm: Pre-training of text and layout for document image understanding. In: Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining. pp. 1192–1200 (2020)

[46] Yang, A., Yang, B., Hui, B., Zheng, B., Yu, B., Zhou, C., et al.: Qwen2 technical report. arXiv preprint arXiv:2407.10670 (2024), qwen Team

[47] Yang, Y., Cheng, W., Chen, S., Zeng, X., Yin, F., Zhang, J., Wang, L., Yu, G., Ma, X., Jiang, Y.G.: Omnisvg: A unified scalable vector graphics generation model. In: The Thirty-ninth Annual Conference on Neural Information Processing Systems (2025)

[48] Zhao, Z., Chen, Y., Hu, Z., Chen, X., Ni, B.: Vector graphics generation via mutually impulsed dual-domain diffusion. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. pp. 4420–4428 (2024)

[49] Zhu, J., Wang, W., Chen, Z., Liu, Z., Ye, S., Gu, L., Tian, H., Duan, Y., Su, W., Shao, J., et al.: Internvl3: Exploring advanced training and test-time recipes for open-source multimodal models. arXiv preprint arXiv:2504.10479 (2025)